Technical introduction to PSOC™ Edge E84 features

Chapter 7: Machine learning

About this document

Scope and purpose

This document is the training manual for the technical introduction to PSOC™ Edge E84 features. This manual helps in getting started with different applications leveraging key features for PSOC™ Edge, using ModusToolbox™ software.

The training is divided into multiple chapters covering topics like audio, graphics, machine learning, among others.

Chapter 7 discusses PSOC™ Edge's machine learning (ML) features.

Intended audience

The document is intended for design engineers, technicians, and developers of electronic systems.

Table of contents

About this document **1**
Table of contents **2**
1 Introduction **3**
2 Prerequisites **4**
2.1 Tools for machine learning labs 4
3 Code example: DEEPCRAFT™ ML data collection (IMU or PDM/PCM) **5**
3.1 Objective 5
3.2 Description 5
3.3 Hardware overview 6
3.4 Project creation 7
3.5 Data collection with DEEPCRAFT™ Studio using Graph UX 8
3.6 Conclusion 15
4 DEEPCRAFT™ Studio: Model creation and generation **16**
4.1 Objective 16
4.2 DEEPCRAFT™ Studio project creation 16
4.3 DEEPCRAFT™ Studio data import 18
4.4 Data labeling and set assignment 20
4.5 Pre-processing 23
4.6 Model generation 24
4.7 Generate model files 28
4.8 Conclusion 30
5 Code example: DEEPCRAFT™ ML deploy (IMU or PDM/PCM) **31**
5.1 Objective 31
5.2 Description 31
5.2.1 Description of movement type detection project 31
5.2.2 Description of baby crying detection project 31
5.3 Hardware overview 32
5.4 Project creation 32
5.5 Output 34
5.5.1 Output using IMU 34
5.5.2 Output using PDM/PCM 34
5.6 Modify the application 35
5.6.1 Further modification 35
5.7 Conclusion 35
6 Appendix A: Creating a PSOC™ Edge application in ModusToolbox™ **36**
7 Appendix B: KIT_PSE84_EVAL details **39**
Revision history **40**
Disclaimer **41**

1 Introduction

This manual provides detailed instructions to create, configure, build, and run several application code examples on the PSOC™ Edge E84 MCU.

2 Prerequisites

See the **Prerequisites** section in the **Chapter 1 training manual** for a complete list of prerequisites and development tools.

2.1 Tools for machine learning labs

Note that machine learning labs utilize the following additional tools:

- DEEPCRAFT™ Studio v5.6.3681 or later
  - Create an Imagimob account to train models

3 Code example: DEEPCRAFT™ ML data collection (IMU or PDM/PCM)

3.1 Objective

To get started with our machine learning (ML) model, we will use a ModusToolbox™ code example to collect data from sensors which can be imported into DEEPCRAFT™ Studio.

This guide supports collecting data from two sources:

- Using an inertial measurement unit (IMU)
- Using a digital microphone via pulse-density modulation to pulse-code modulation (PDM/PCM)

Note: Images and steps in this section are shown for IMU if the process between both flows is similar; otherwise, the guide explains the differences when using PDM/PCM.

3.2 Description

The code example used in this guide demonstrates how to collect data by implementing the DEEPCRAFT™ streaming protocol v2, allowing the streaming of sensor data and other information from the PSOC™ Edge EVK into DEEPCRAFT™ Studio over USB.

The code example supports collecting data from different sources using the Cortex® M55 (CM55) CPU, including:

- Inertial measurement unit (IMU-BMI270),
- Magnetometer (BMM350),
- XENSIV™ digital MEMS microphones (IM73D122V01) using PDM/PCM,
- XENSIV™ digital barometric air pressure sensor (DPS368),
- Digital humidity and temperature sensor (SHT40T), and,
- XENSIV™ 60 GHz radar sensor (BGT60TR13C)

This guide will provide the steps to use IMU and microphone using PDM/PCM.

Data is transmitted over USB and will be used in subsequent steps to generate a model.

When using the IMU:

- The code example collects data from the IMU BMI270 included in the PSOC™ Edge EVK
- Data consists of the 3-axis accelerometer and 3-axis gyroscope data obtained from the IMU
- Data is read from IMU over I2C and then transmitted to the PC over USB
- The following configurations are supported by DEEPCRAFT™

Table 1 Supported configurations for IMU

Configuration	Ranges/Options
Frequency	50 Hz, 100 Hz, 200 Hz, 400 Hz
Accelerometer	2 G, 4 G, 8 G, 16 G
Gyroscope	125 dps, 250 dps, 500 dps, 1000 dps, 2000 dps
Mode	Combined, Split, Only Accelerometer, Only Gyroscope
Configuration	Ranges/Options
Frequency	50 Hz, 100 Hz, 200 Hz, 400 Hz

When using the microphone:

- Data is collected from a digital microphone included in the PSOC™ Edge EVK using PDM to PCM
- Data is sampled at the rate configured in DEEPCRAFT™ Studio and transmitted over USB

Table 2 Supported configurations for PDM/PCM

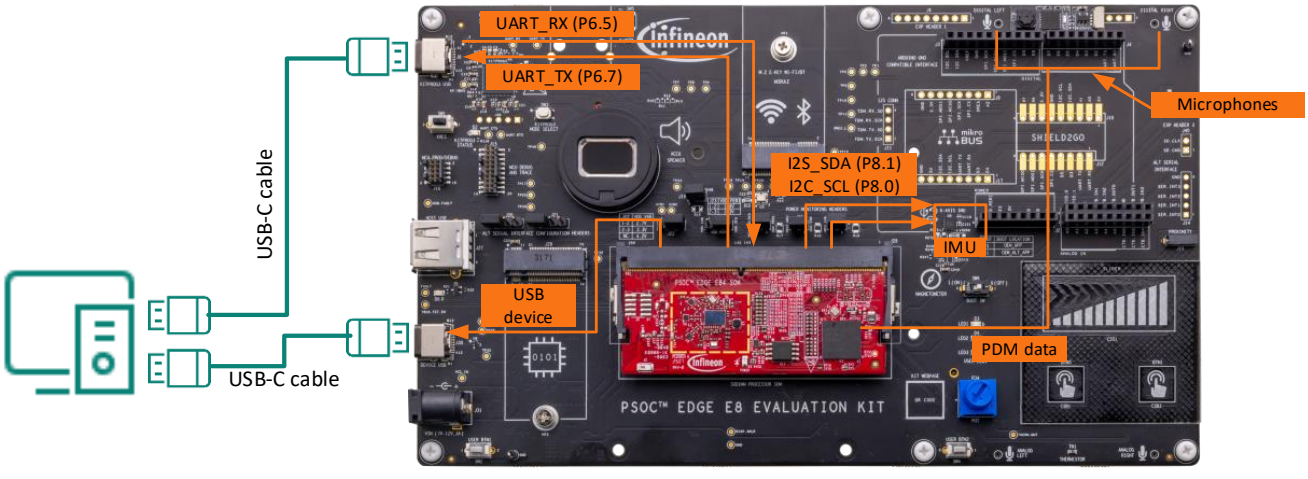
Configuration	Ranges/Options
Gain	83 dB , 77 dB , 71 dB , 65 dB , 59 dB , 53 dB , 47 dB , 41 dB , 35 dB , 29 dB , 23 dB , 17 dB , 11 dB , 5 dB , -1 dB , -7 dB , -13 dB , -19 dB , -25 dB , -31 dB , -37 dB , -43 dB , -49 dB , -55 dB , -61 dB , -67 dB , -73 dB , -79 dB , -85 dB , -91 dB , -97 dB , -103 dB
Frequency	8 kHz, 16 kHz, 22.05 kHz, 44.1 kHz, 48 kHz
Stereo (Mode)	Yes, No

3.3 Hardware overview

- UART (transmit data)
  - P6_7 (UART_TX)
  - P6_5 (UART_RX)
- I2C (IMU)
  - P8.1 (SDA)
  - P8.0 (SCL)
- PDM/PCM (Audio)
  - P8_6 (PDM_Data)
  - P8_5 (PDM_CLK)
- USB device

Note: This example requires BOOT SW in ON position.

See Appendix B: KIT_PSE84_EVAL details for more details.



Note: the signals shown are for illustration purposes, not exact routing

Figure 1 Hardware diagram for ML data collection example

3.4 Project creation

1. Follow steps 1-4 in Appendix A: Creating a PSOC™ Edge application in ModusToolbox™ to create a new application.
2. When creating the application, select the **PSOC™ Edge Machine Learning DEEPCRAFT Data Collection** application under the **Machine Learning** section.

   Note: To prevent issues with Windows path length, it's recommended to rename the project if your worskpace path is long.

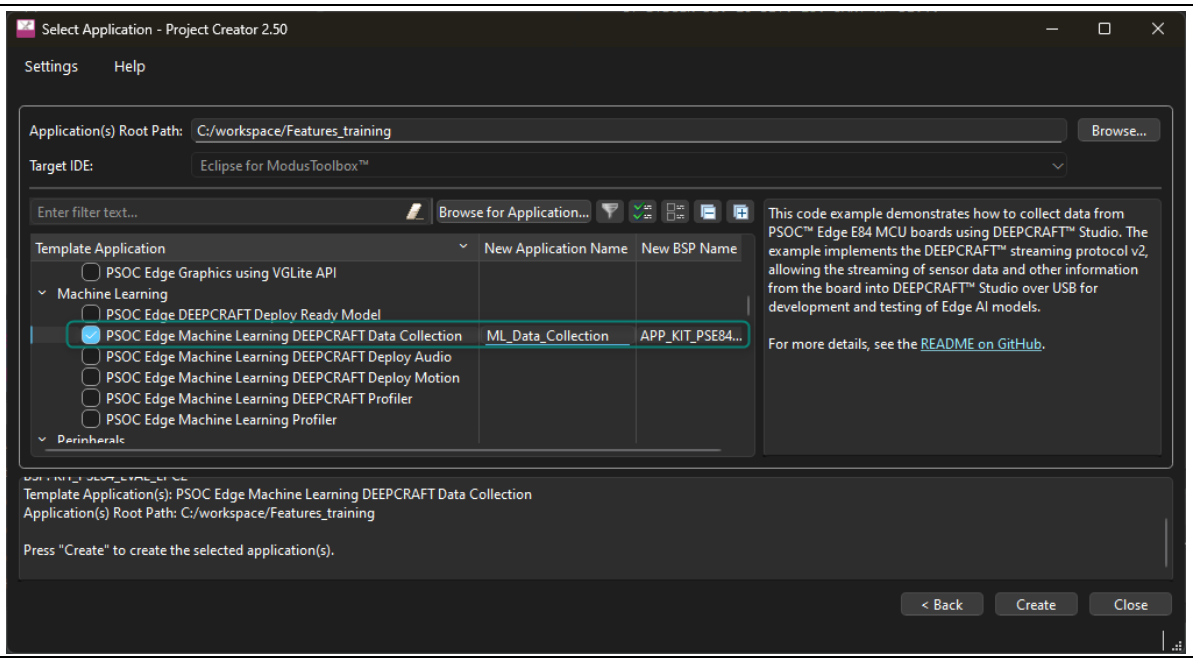


Figure 2 Creation of machine learning data collection example

3. Build and program the application.
4. The application will start running and you should observe the following output on a terminal connected to the kitProg USB (J8).

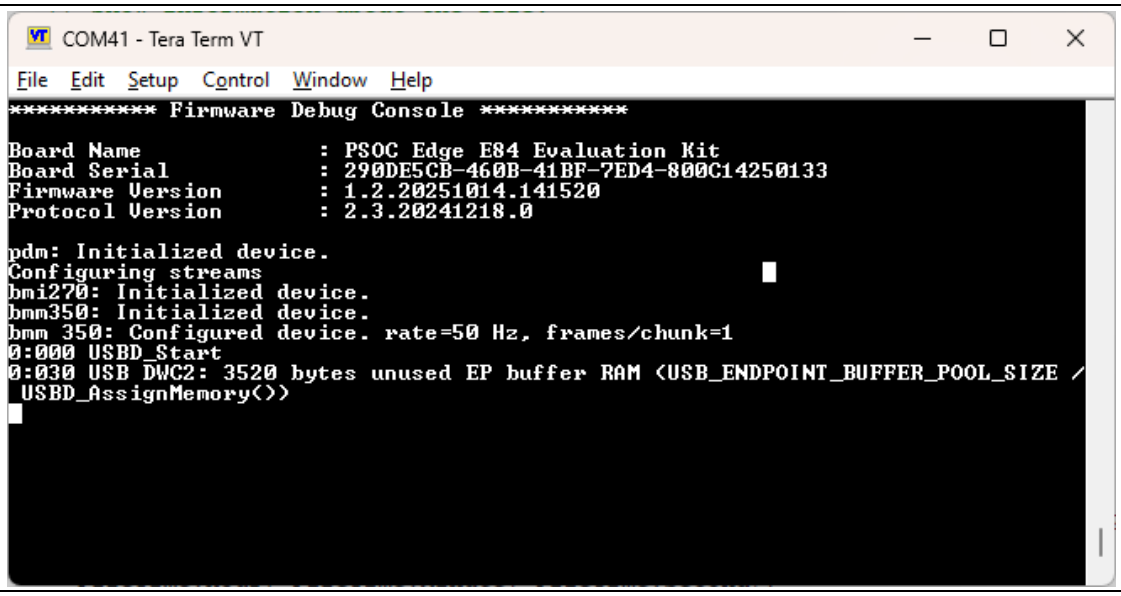


Figure 3 Terminal output of ML data collection example

5. Connect a second USB cable to the USB device port available at J30.
   This will enable the serial port for sensor data collection, and the following will be shown in the terminal.
   Note: The terminal should be connected to the kitProg USB port, not to the USB device port, but it shows then a USB host is connected to the PSOC™ Edge USB device interface.

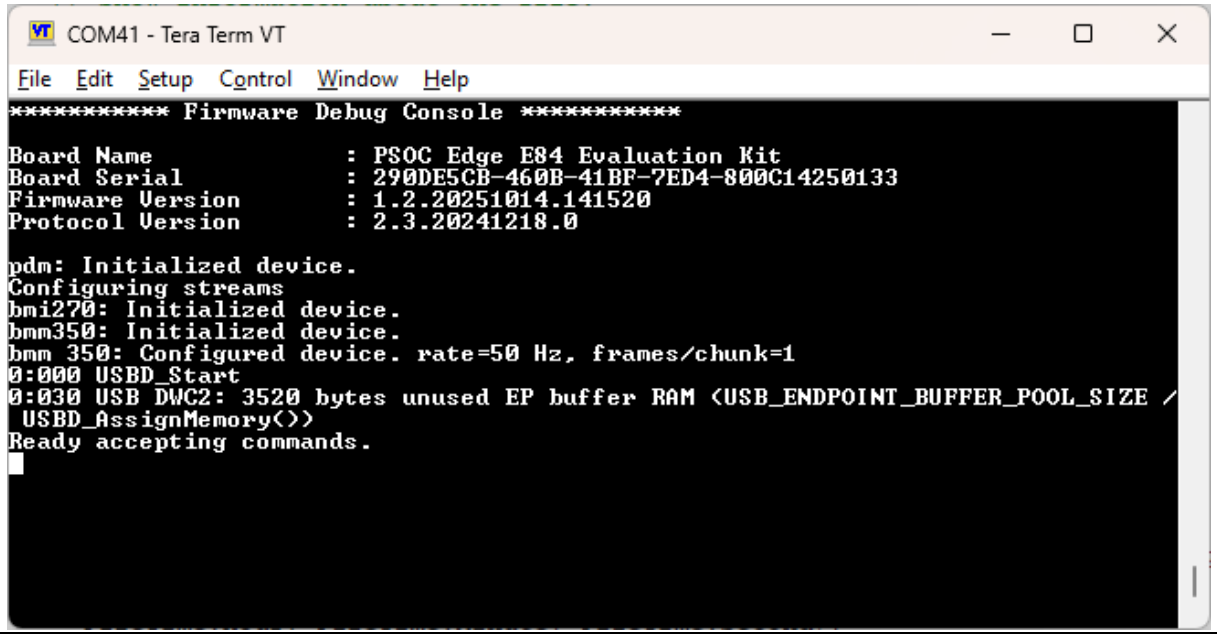


Figure 4 Terminal output after connecting USB device port

The device is ready to transmit data to DEEPCRAFT™ Studio.

3.5 Data collection with DEEPCRAFT™ Studio using Graph UX

DEEPCRAFT™ Studio includes Graph UX, an intuitive interface used to visualize end-to-end machine learning workflow as graphs. We will be using Graph UX to perform live data collection using the IMU or microphone.

6. Open DEEPCRAFT™ Studio and start a new project by clicking on **File** > **New Project** or clicking the **New Project** button in the Welcome window.

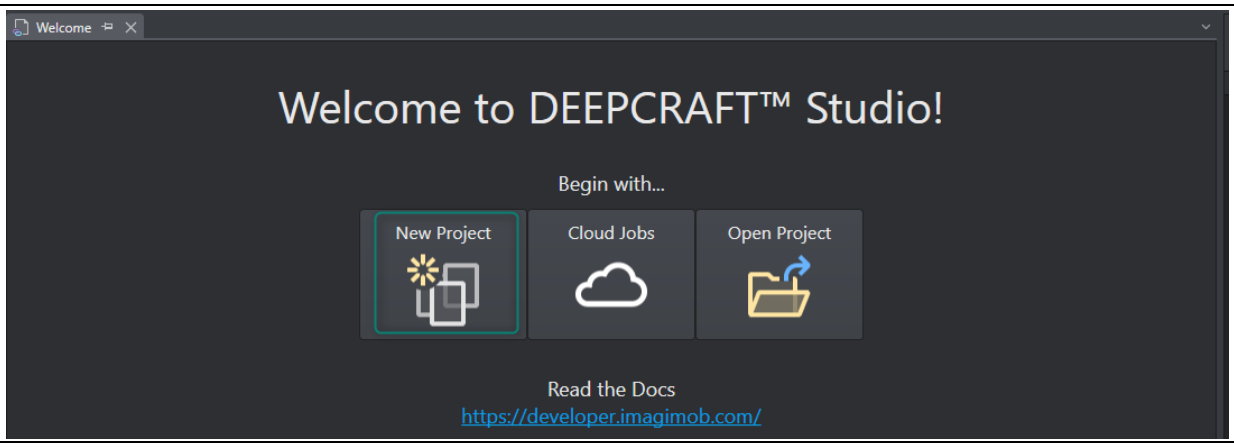


Figure 5 Start a new project in DEEPCRAFT™ Studio

7. Select the **Templates** > **Graph UX** > **Generic** folder, click on **Empty Project**, add a **New Project Name** of your preference, and select a workspace **Location**.

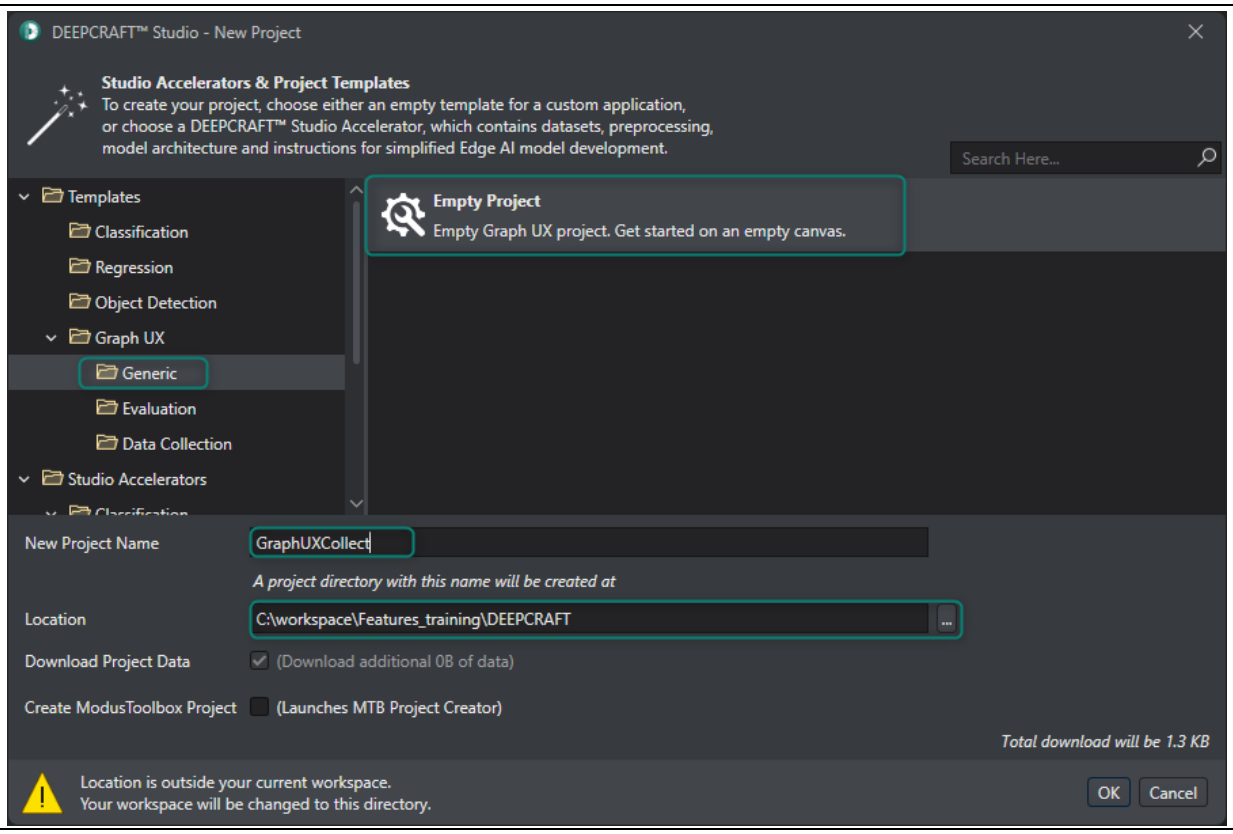


Figure 6 Start a Graph UX project

8. With the EVK programmed and running the DEEPCRAFT™ machine learning data collection example as explained earlier in this section, open the canvas *Main.imunit* and expand the **Boards** folder in the **Node explorer** panel.

   Observe how DEEPCRAFT™ Studio recognizes the PSOC™ Edge EVK and lists the sensors supported by this board.

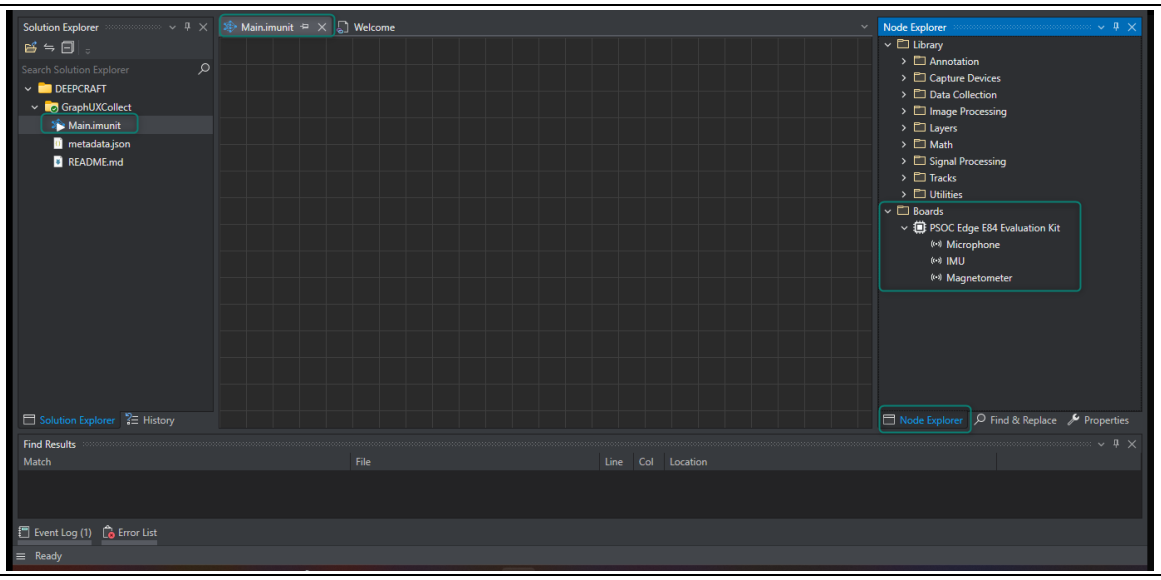


Figure 7 Graph UX blank canvas showing PSOC™ Edge E84 EVK sensors

9. Drag-and-drop one of the corresponding sensors into the canvas.

a) For IMU project, select the **IMU** component.
The sensor is preconfigured to 50Hz frequency, 8G range, 500dps and Combined mode, which is how the deploy project in the next steps expects it.
Note that the deploy example only expects accelerometer data without gyroscope.

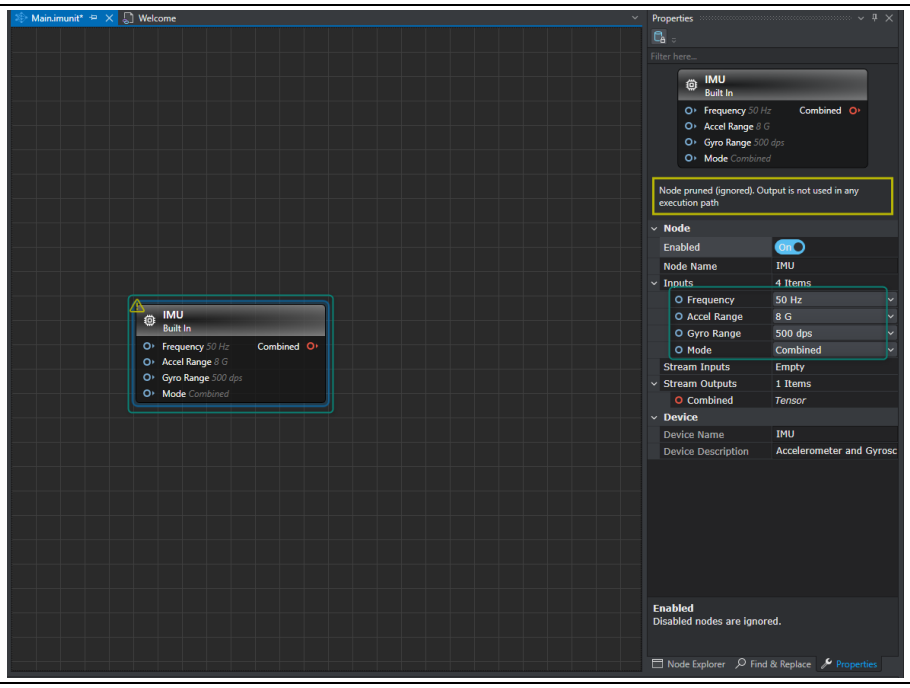


Figure 8 Adding IMU sensor to Graph UX

b) For PDM/PCM project, select the **microphone** component.
The sensor is preconfigured to 16kHz, 5dB gain, and mono mode, which is how the deploy project in the next steps expects it.

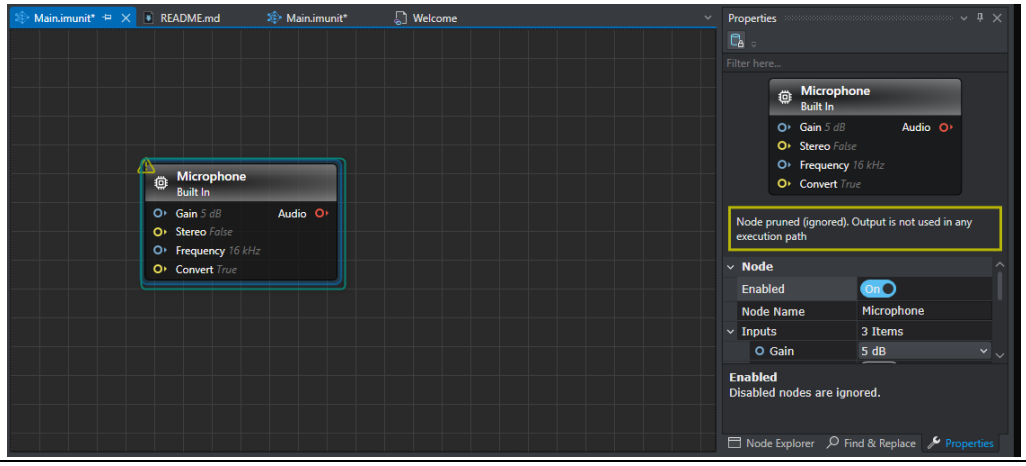


Figure 9 Adding microphone sensor to Graph UX

10. Streaming video data is not necessary to build a model, but it can be helpful in labeling data. Go to the **Node Explorer** and drag-and-drop the **Library** > **Capture Device** > **Local Camera** component.
Note that DEEPCRAFT™ Studio allows to select the corresponding camera device if multiple cameras are connected to your PC.

a) For IMU project:

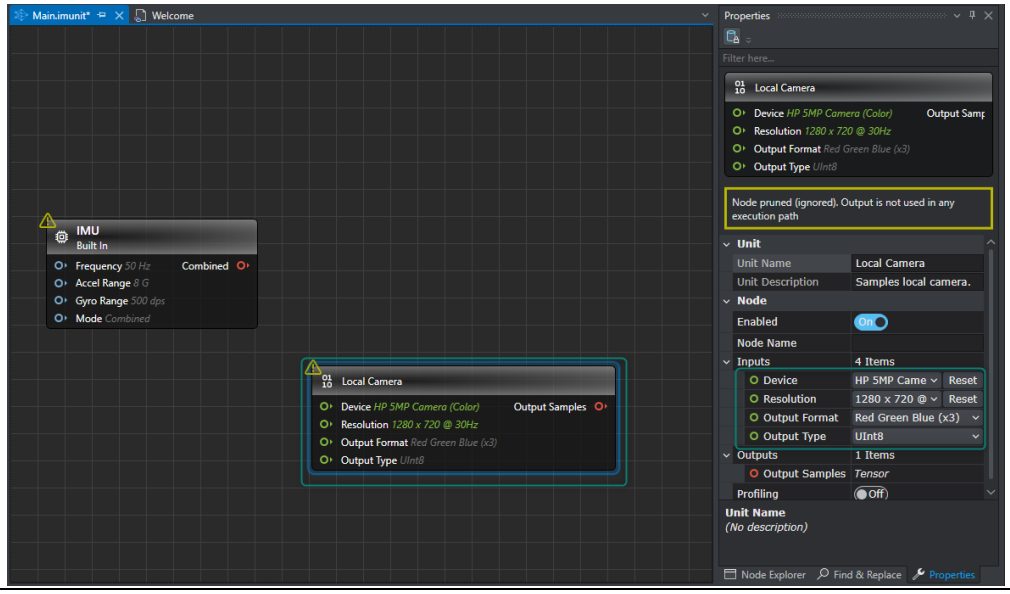


Figure 10 Adding camera to Graph UX

b) For PDM/PCM project, we will skip this step since capturing video is not as useful when labeling data.

11. Visualization nodes help visualize data collected from sensors in the form of tracks in the session file. Drag-and-drop corresponding **Tracks** from **Node Explorer** inside the **Library** > **Tracks** folder.

a) For IMU, drag-and-drop a **Data Track** and connect it to **IMU**; and a **Video Track** which must be connected to the **Local Camera** component.

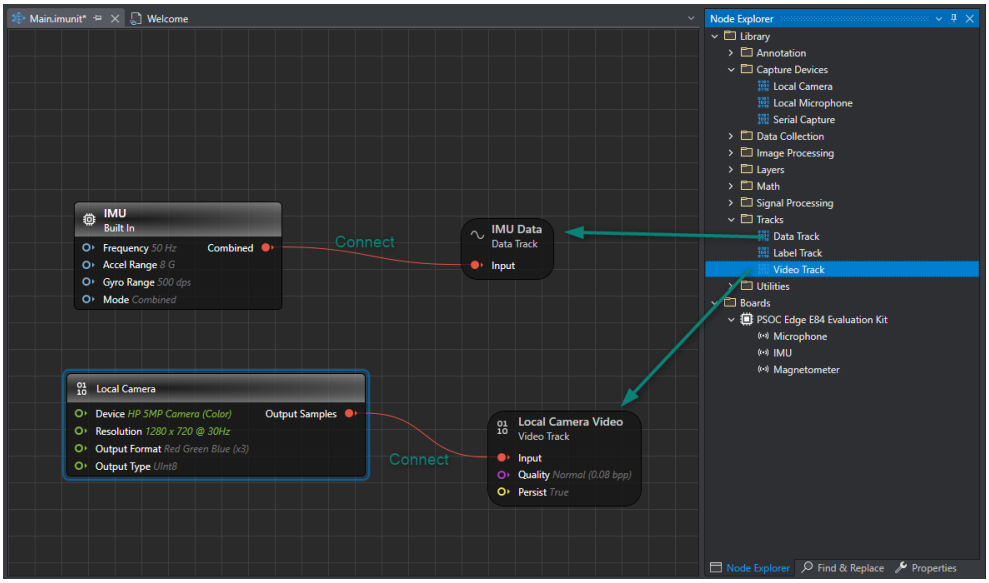


Figure 11 Adding IMU tracks to Graph UX

b) For PDM/PCM, drag-and-drop a **Data Track** and connect it to the **Microphone** component.

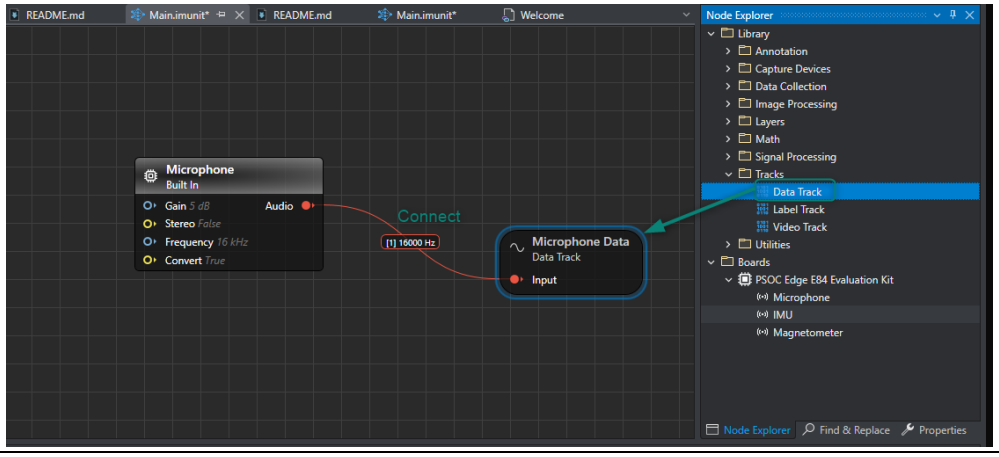


Figure 12 Adding microphone tracks to Graph UX

12. We can label real-time data by adding a **Predefined Labels** component into the canvas. From the **Node Explorer**, drag-and-drop **the Library** > **Annotation** > **Predefined Labels** component into your canvas. Then, select it and label.

    a) For IMU, add the labels: **shaking and circle.** These labels are expected by the machine learning model used in the next steps.

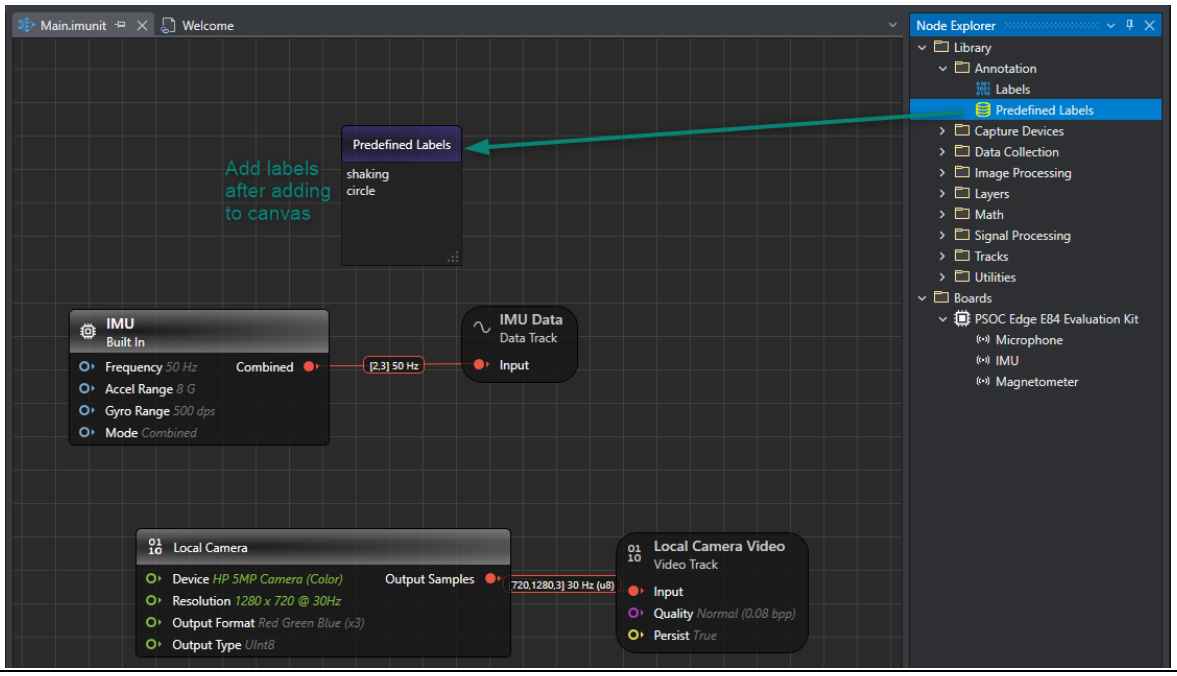


Figure 13 Adding predefined IMU labels in Graph UX

    b) For PDM/PCM, add the label **baby_cry**. This label is expected by the machine learning model utilized in the next steps.

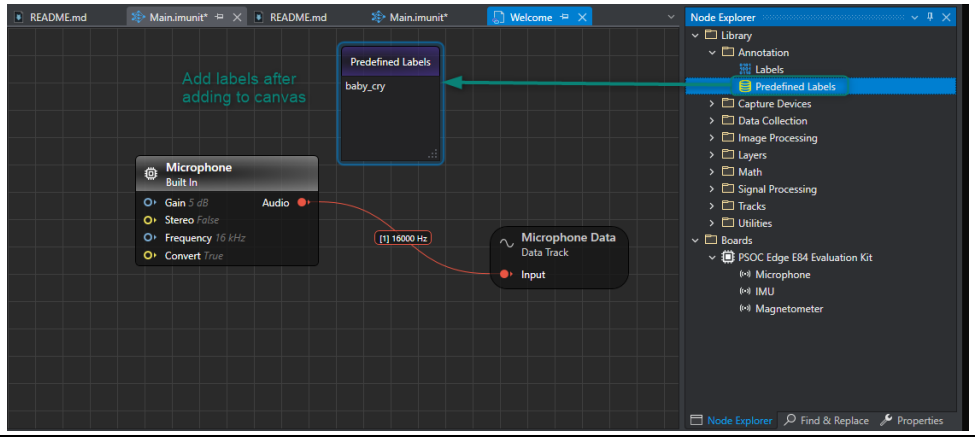


Figure 14 Adding predefined PDM/PCM labels in Graph UX

13. We are now ready to collect data. Click the **Start** button to start the Graph UX live recording session.

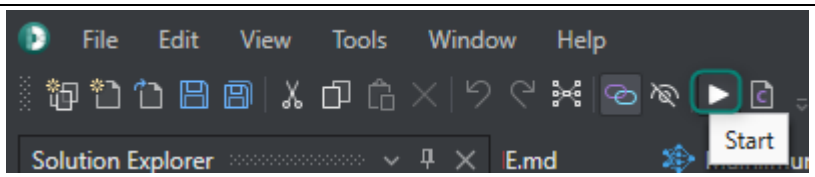


Figure 15 Start Graph UX live recording session

14. In the newly opened **live.imsession** window, click the **Start** button and observe the data. Select a predefined label to make it easier to detect each activity.

    Note: You can use Alt+number to select different labels.

    a) For IMU, perform the movement activities: **shaking** or **circle**.

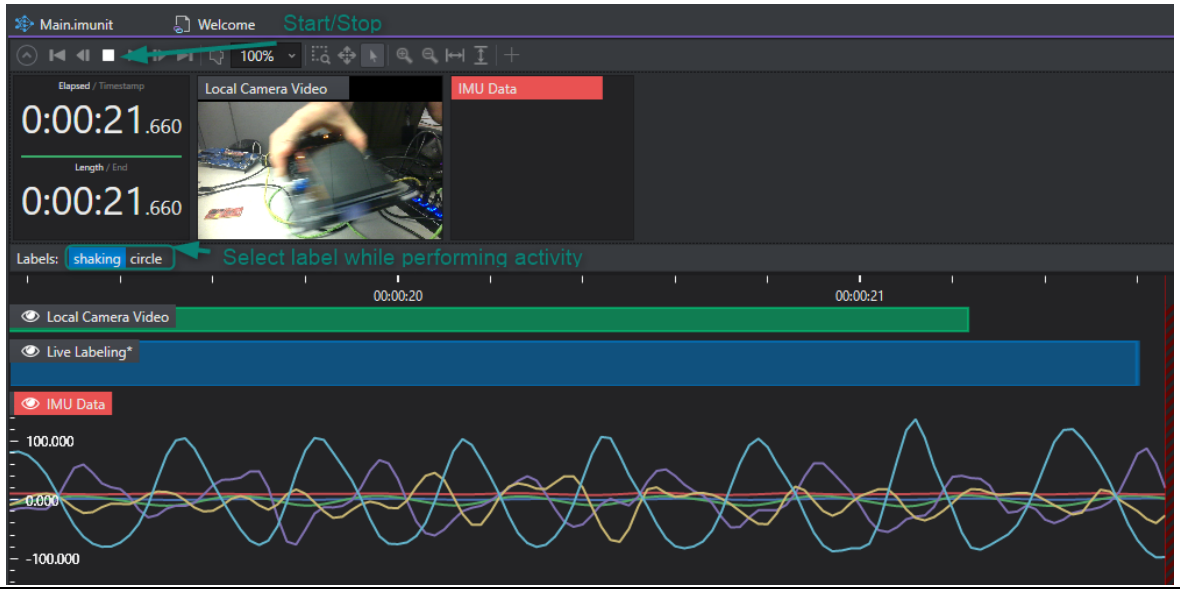


Figure 16 Recording and labeling live IMU data on Graph UX

b) For PDM/PCM, play baby-crying sounds and click on the **baby_cry** label when sound is playing.

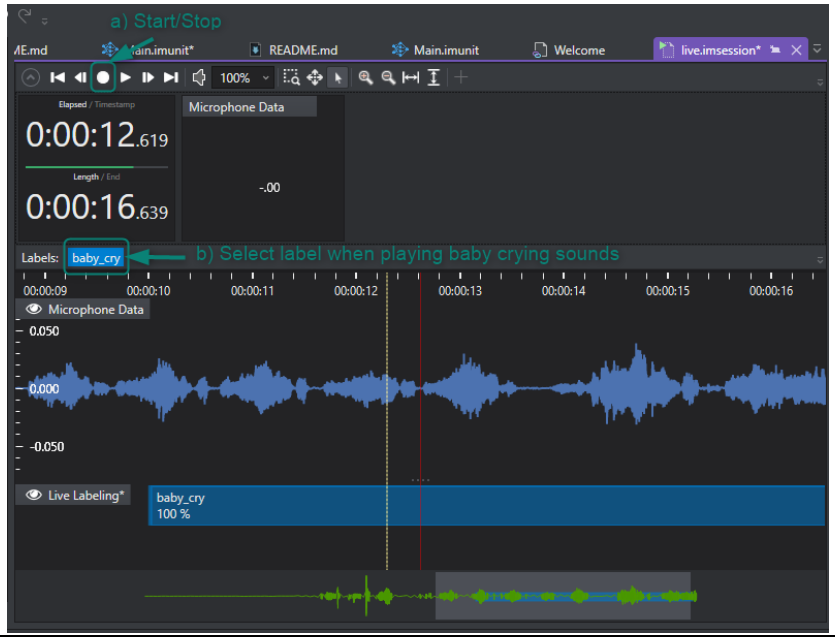


Figure 17 Recording and labeling live PDM/PCM data using Graph UX

15. Click **Ctrl+S** or **File** > **Save**. A window will appear with information about the session. Select a **location**, **Session Name**, and modify **Track Options** if needed.

a) For IMU, you should observe tracks for label, video and data:

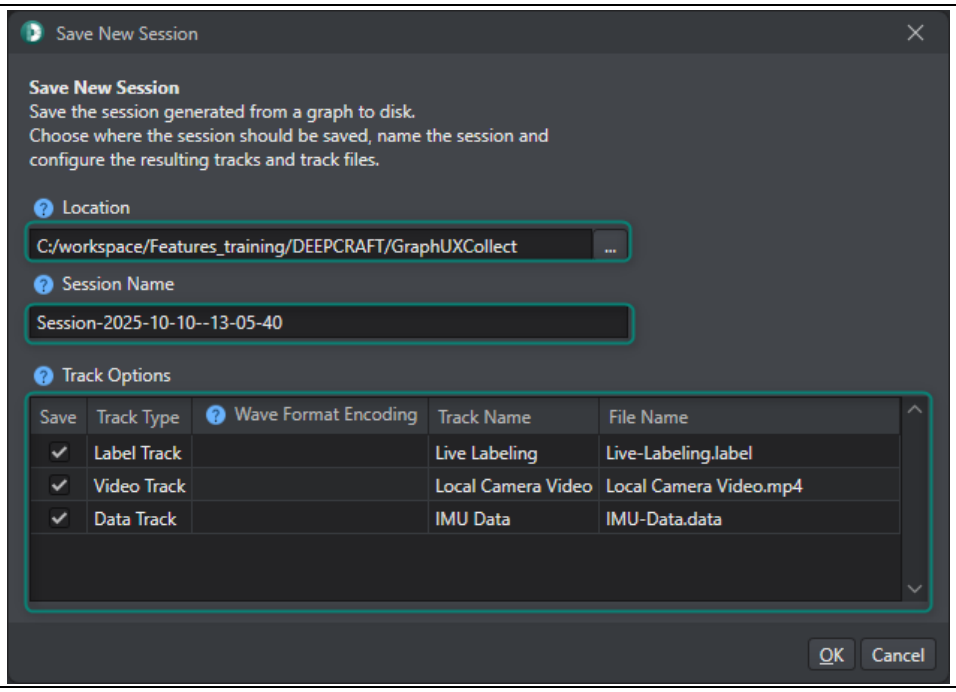


Figure 18 Saving a live recording session for IMU

b) For PDM/PCM, you should observe tracks for audio data and label:

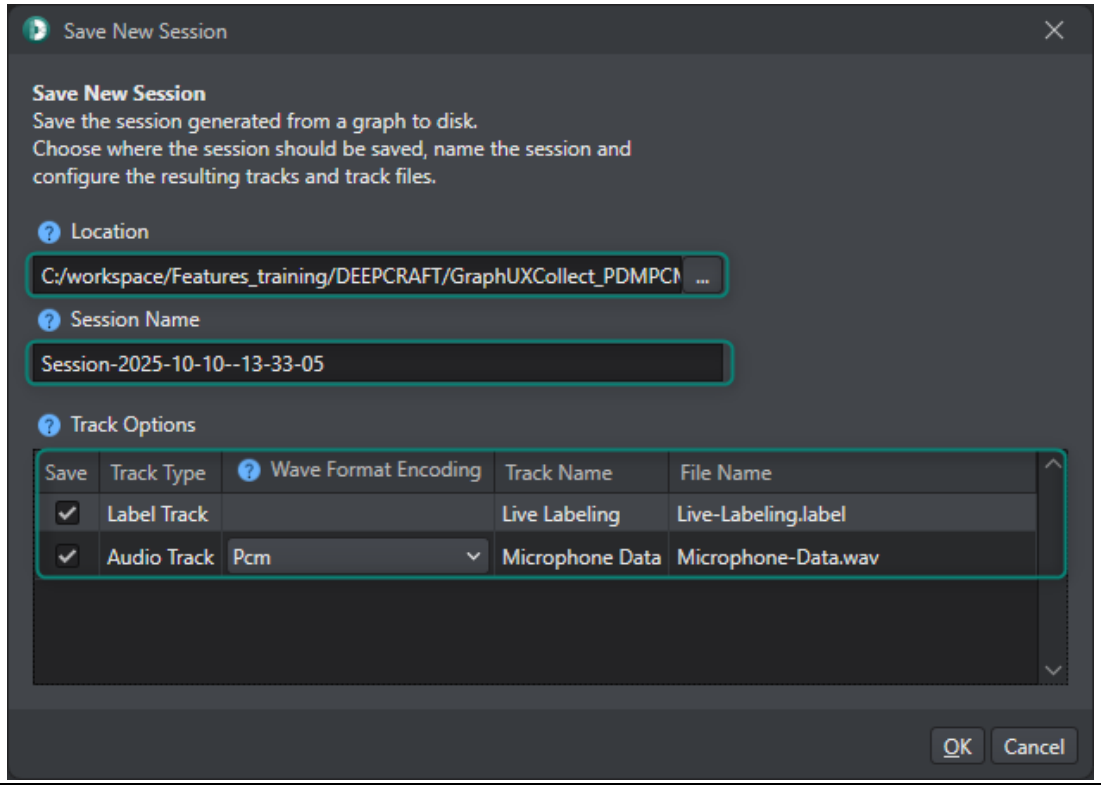


Figure 19 Saving a live recording session for PDM/PCM

16. Repeat the process to record more data.

These output files will be imported into DEEPCRAFT™ Studio.

3.6 Conclusion

We successfully created a PSOC™ Edge E84 MCU application using ModusToolbox™ to collect data and transmit it to DEEPCRAFT™ Studio using Graph UX.

You should now have IMU and/or PDM/PCM data which will be used to train a machine learning model using DEEPCRAFT™ Studio.

4 DEEPCRAFT™ Studio: Model creation and generation

4.1 Objective

In the previous section, we collected data to detect either different motion activities using IMU, or for baby crying detection using a microphone via PDM/PCM. In this section, we will go through the steps of creating a model, data labeling, pre-processing, model generation, validation, and deployment using the data collected in the previous example.

Note: Images and steps in this section are shown for IMU if the process between both flows is similar; otherwise, the guide explains the differences when using PDM/PCM.

4.2 DEEPCRAFT™ Studio project creation

DEEPCRAFT™ Studio provides different templates for custom applications and accelerators, which contain datasets, pre-processing, model architecture, and instructions for simplified Edge AI model development. In this session, we will use the **DEEPCRAFT™ Studio Accelerators** to detect movement types from IMU data and baby cry detection from audio data.

Follow the next steps to create the DEEPCRAFT™ Studio machine learning project:

1. In DEEPCRAFT™ Studio, start a new project by clicking on **File** > **New Project** or clicking the **New Project** button in the Welcome window.
2. Select the corresponding accelerator project in the **New Project** window, provide a **New Project Name,** and select the **Location.** Make sure **Download Project Data** is selected.

   a) For IMU, select Studio Accelerators > Classification > IMU and Vibration > Movement Type Detection

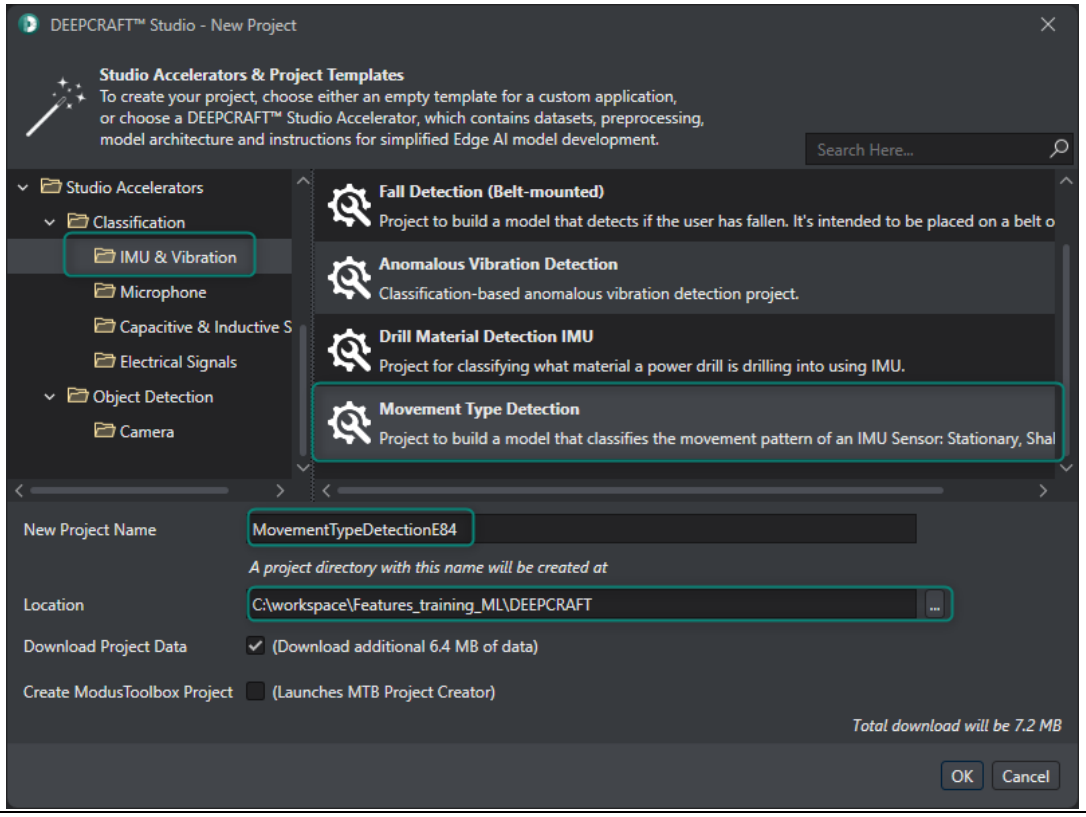


Figure 20 Creating DEEPCRAFT™ Studio project for movement type detection using IMU

b) For PDM/PCM, select **Studio Accelerator** > **Classification** > **Microphone** > **Baby Cry Detection (Studio Accelerator)**

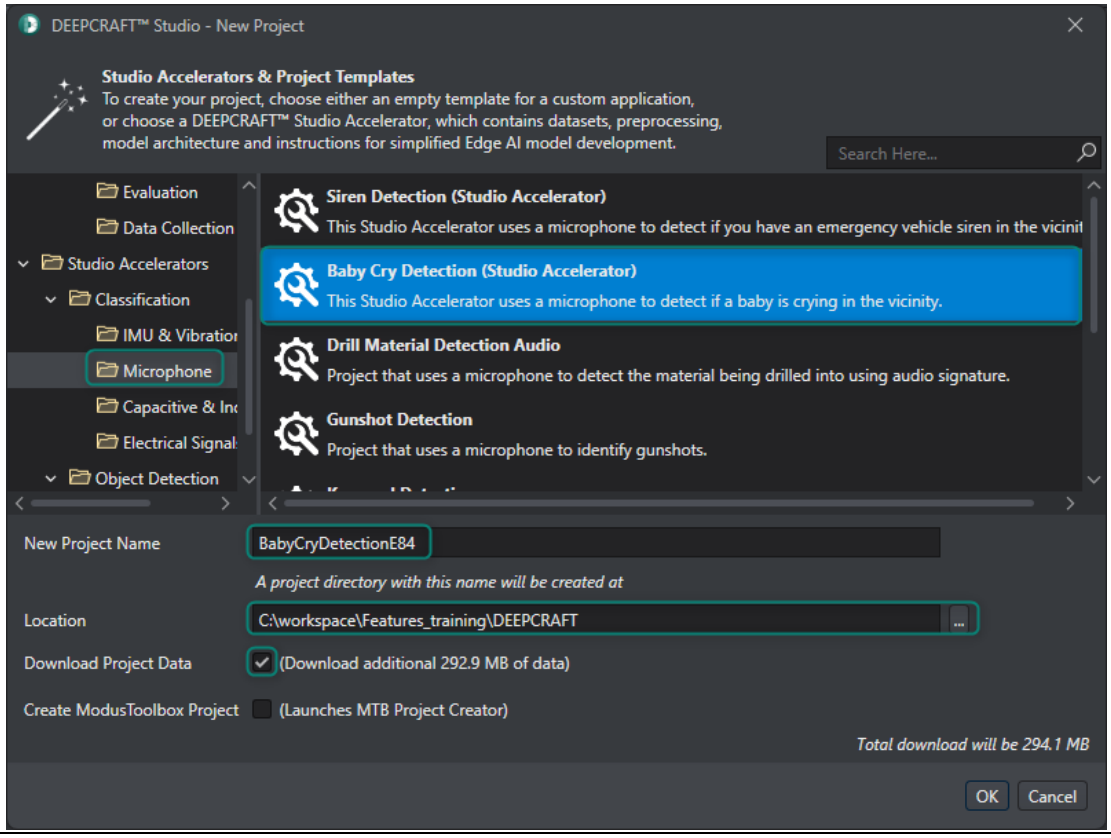


Figure 21 Creating DEEPCRAFT™ Studio project for baby cry detection using PDM/PCM

3. Observe that DEEPCRAFT™ Studio will download data in the background. This is an essential step since adding more data enables the project to generate a more accurate model. Click **OK** to close the window.

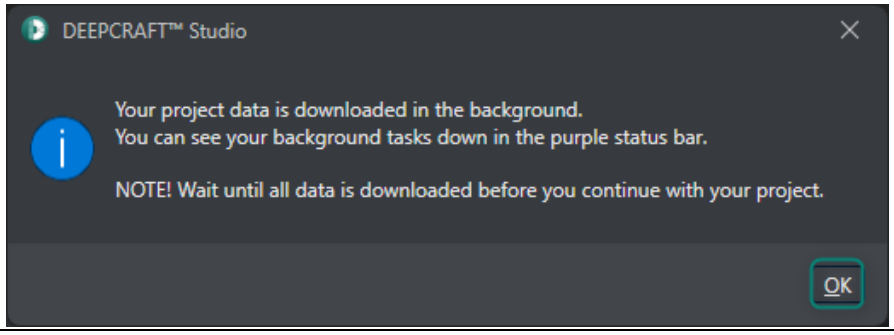


Figure 22 Data download in DEEPCRAFT™ Studio

The project will be created, and we can proceed with importing the collected data.

4.3 DEEPCRAFT™ Studio data import

DEEPCRAFT™ Studio allows for easy importing of new data into a project. DEEPCRAFT™ Studio supports importing the file types in the following table.

Table 3 Data formats supported in DEEPCRAFT™ Studio

Data type	File type
Audio	.wav
Data	.csv, .data, .label
Video	.mp4

Follow the next steps to import the previously collected data into DEEPCRAFT™ Studio.

1. Open the corresponding project file (.improj) present in the Solutions Explorer tab.

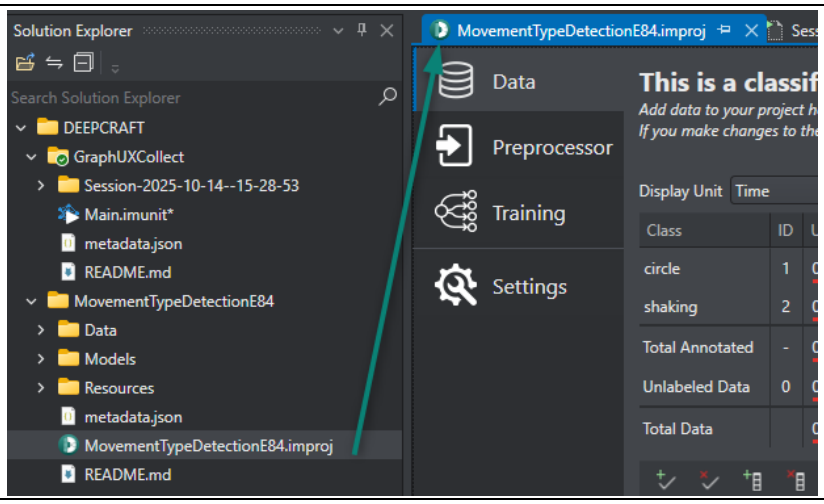


Figure 23 Opening the .improj project file for IMU

*Note: Select the **Data** tab and observe that the project already contains data to make it easier to generate a more accurate model. Double-click the files and observe the data. All imported data is labeled and classified.*

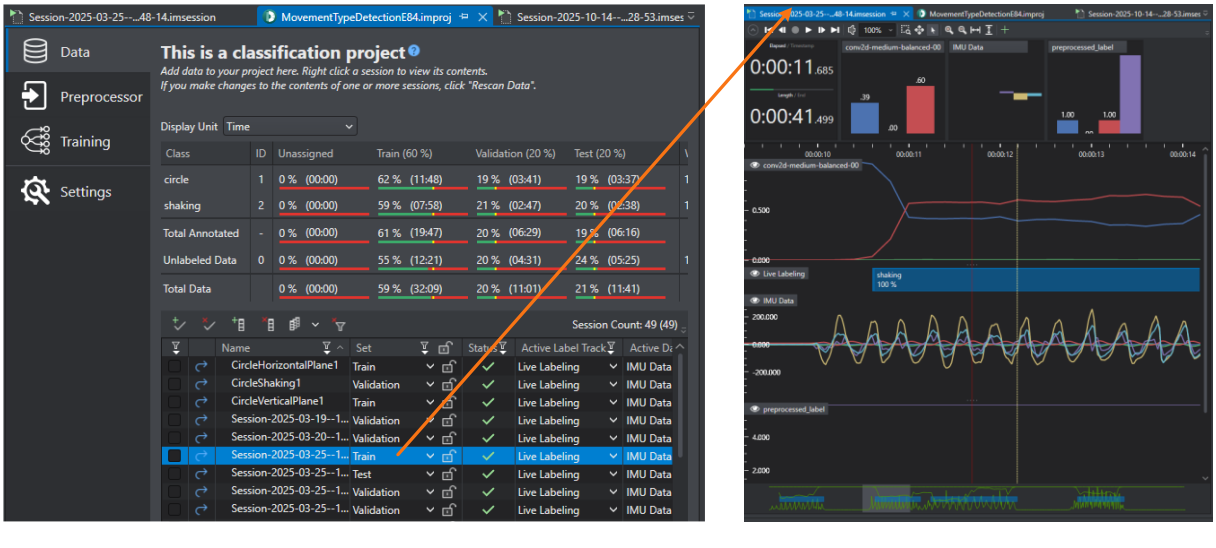


Figure 24 Existing data for IMU project

2. In the **improj** panel, go to the **Data** tab and click the **Add Data** button.

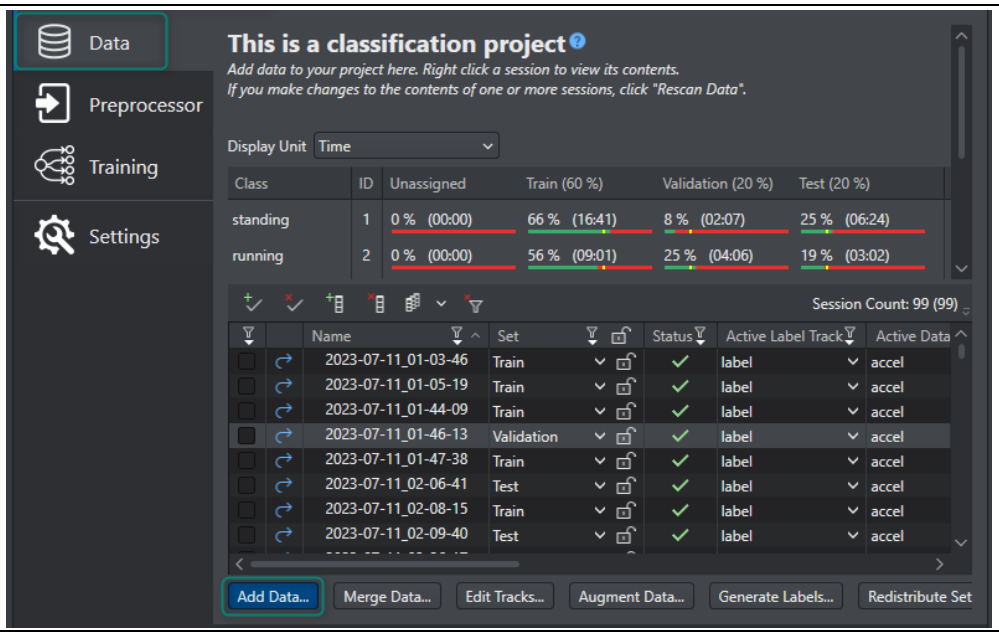


Figure 25 Add data to IMU project

3. In the **Add Data** window, select the folder where the recorded data was stored. DEEPCRAFT™ Studio will show the number of detected tracks. Click **Next**.

   Note: The image below shows steps for IMU which includes tracks for data, label, and optionally video if a camera was used. PDM/PCM data only includes the data and label tracks.

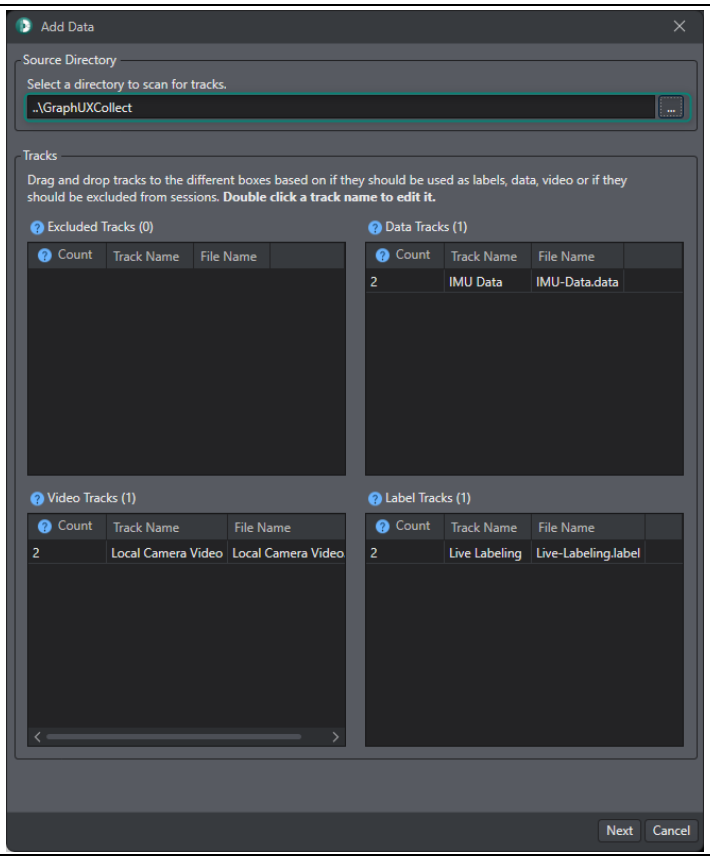


Figure 26 Adding data for IMU

4. A new window will appear with a summary of the data. Click **OK.**

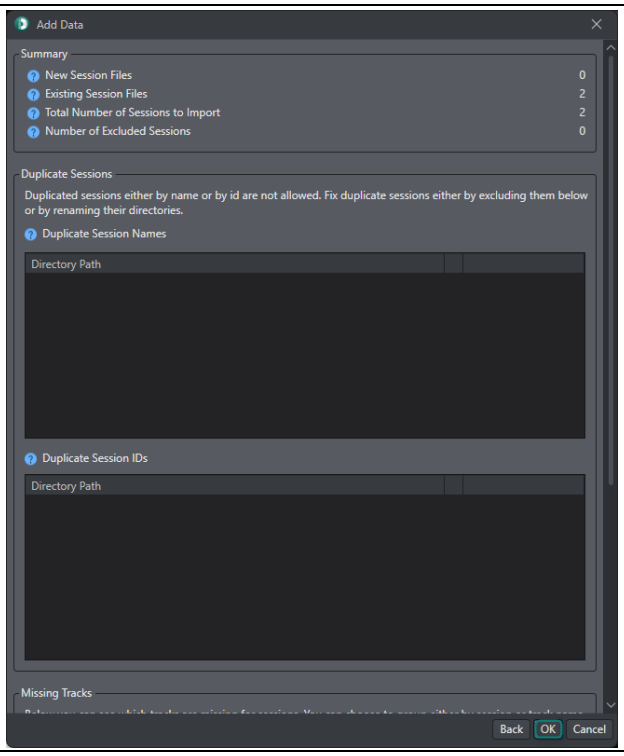


Figure 27 Data summary

4.4 Data labeling and set assignment

Data labeling allows labeling of raw data with expected classifications. When the model is being trained, the data is used, and the model gives a classification, which is then compared to the label given by the user. The model weights are then adjusted depending on whether the correct classification was achieved.

1. We previously labeled our data while collecting; however, DEEPCRAFT™ Studio allows adding and adjusting labels after data is imported.

   To observe and adjust or add labels of the newly imported data, follow the next steps:

   a) In the data window, click on the **Set** filter, and select **Unassigned**. This will allow us to easily observe the newly added data.

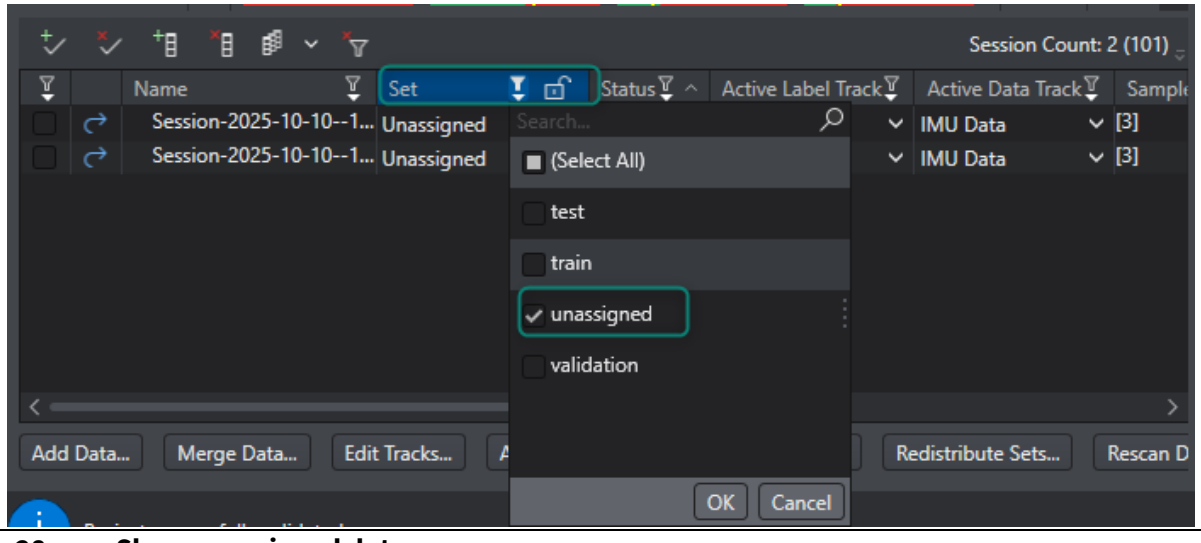


Figure 28 Show unassigned data

b) Double-click in any data entry.

c) This will open the corresponding session, and the data and labels can be observed.

d) Labels can be adjusted simply by dragging them to match the sensor data; or, they can be added by right-clicking on the data window and selecting **Add label**.

e) When adding a new label, the following window will appear

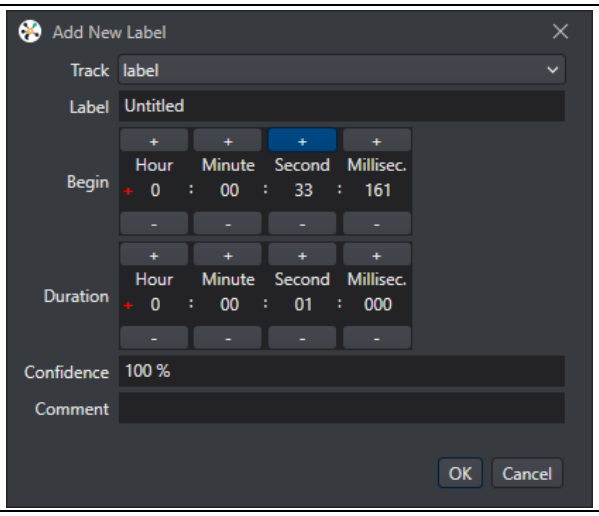


Figure 29 Adding a new label

f) Configure the following parameters:

Table 4 Parameters for new labels

Parameter	Description
Track	Select the label track in which you want to add the labels
Label	Enter the name of the label to add for that specific piece of data in the track
Begin	Set the timestamp from where the label starts
Duration	Set the timestamp for how long the label lasts
Confidence	Enter the confidence percentage for the input label
Comment	Comment for label, if required

*Note: For IMU, the supported labels are: **circle and shaking**; For PDM/PCM, only the **baby_cry** label is implemented.*

g) After a new label is created, it can then be manually adjusted by dragging the sides to make it longer or grabbing the middle of the label to move it.

2. After labeling all the data, go back to the project file (*.improj). Select **Save All**, then select **Rescan Data**, and then **Quick Scan** to ensure all labeling is registered by the project as shown in Figure 30.

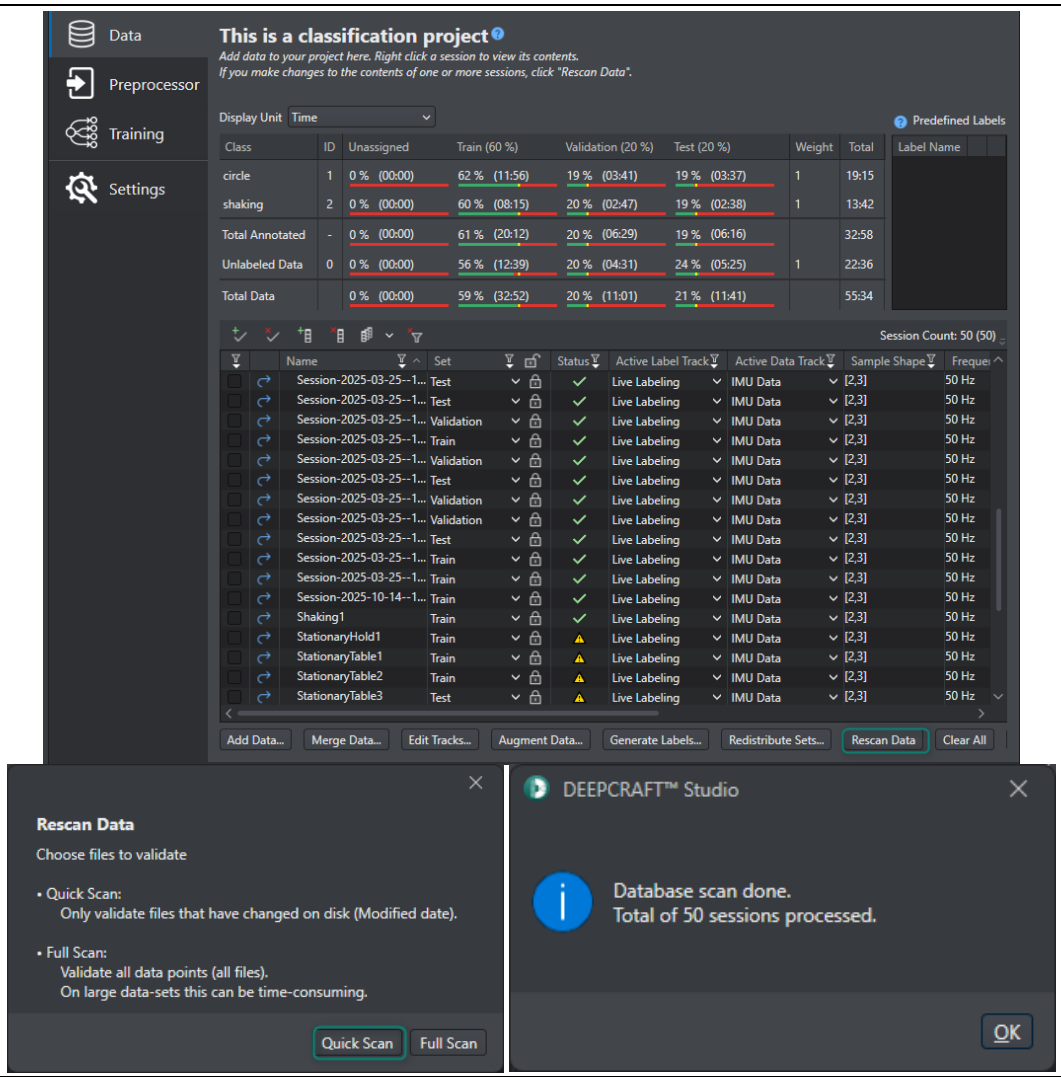


Figure 30 Rescanning Data

3. For the newly added data, select the drop-down menu under **Set** and set to **Train**.
   This sets the data as part of the training data set.

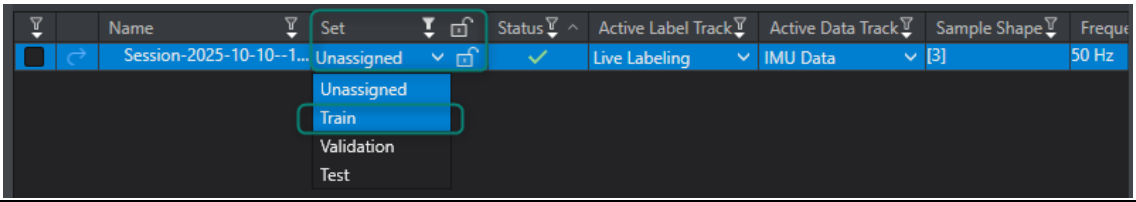


Figure 31 Assigning data to Training set

4. A recommended distribution is: 60% train, 20% validation, 20% test.
   The **Redistribution Sets** button can be used to redistribute collected data.

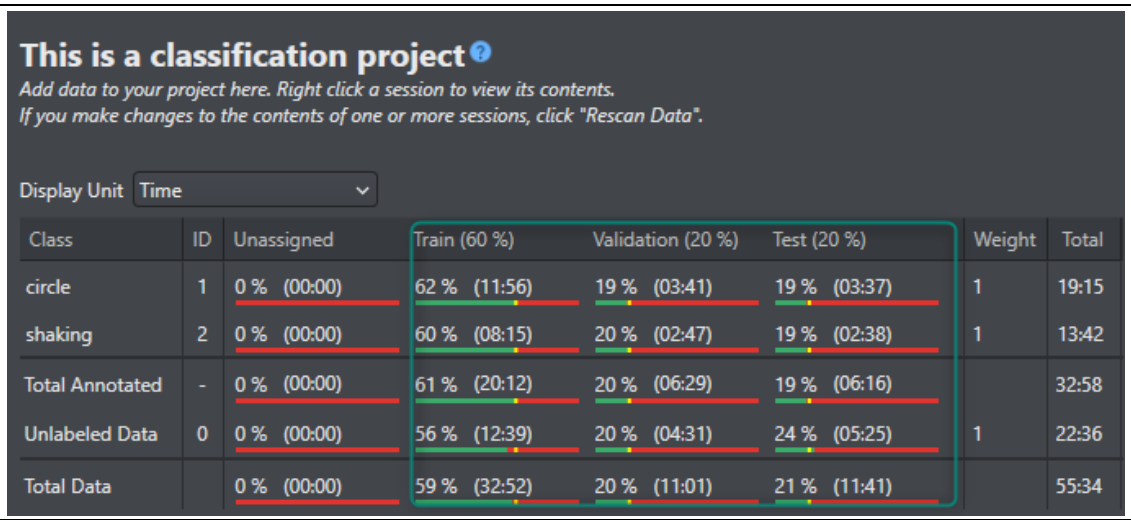


Figure 32 Set distribution

After the data is completely labeled and distributed, it can be used to train the model.

4.5 Pre-processing

DEEPCRAFT™ Studio includes pre-processing that can be configured depending on the type of data being used. For this lab, the pre-processing pipeline is already set in the starter projects. It is encouraged to look at the preprocessor to get familiar with the process of adding layers.

1. To observe the pre-processing layers, select the **Preprocessor** tab.
   a) For IMU, the preprocessor uses a 50-frame sliding window for evaluating data over a period of 2 seconds as the data rate is 50 Hz.

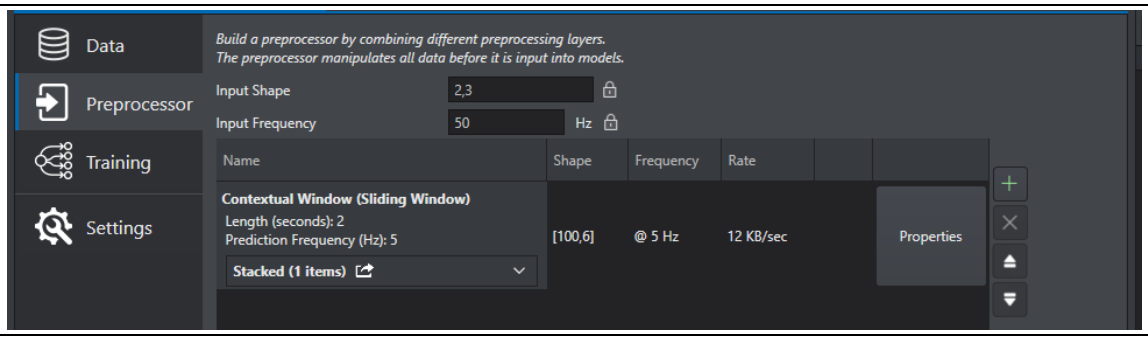


Figure 33 Preprocessor layers for movement type detection project

   a) For PDM/PCM, the preprocessor includes the following layers: sliding window, Hann smoothing, real discrete Fourier transform, Frobenuis norm, Mel filterbank, clip, logarithm, and a second sliding window as shown in Figure 35.

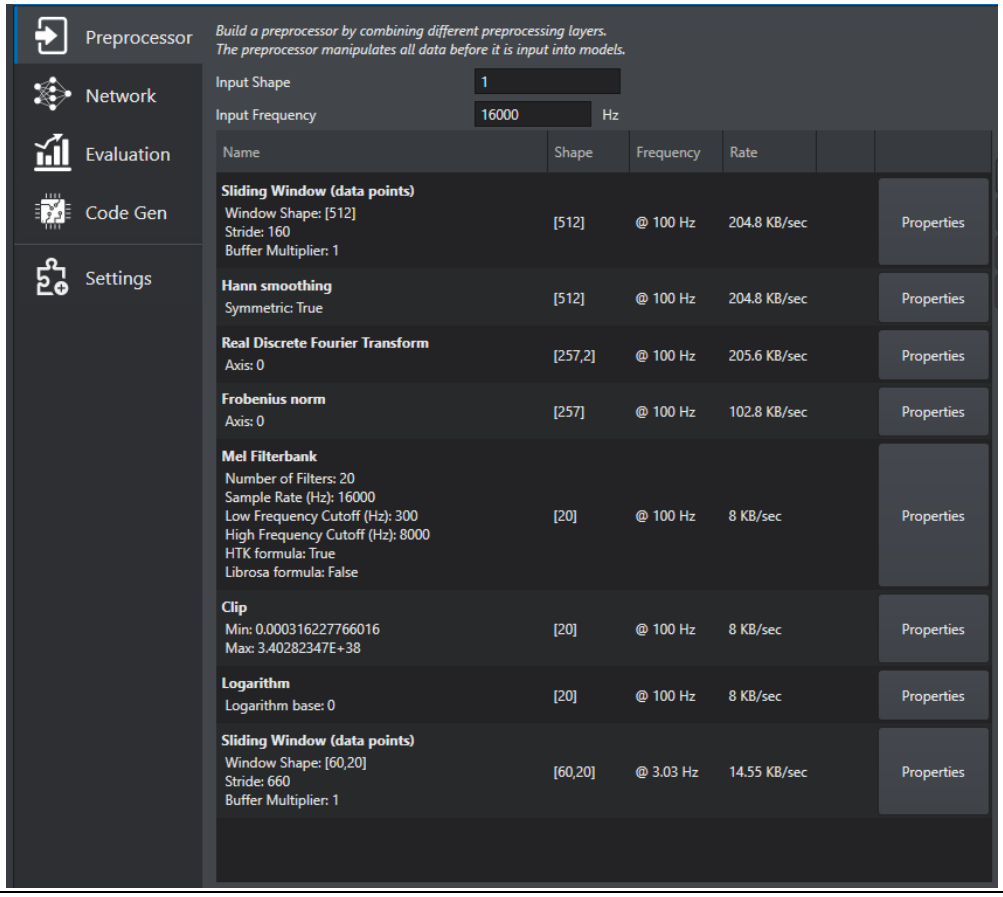


Figure 34 Preprocessor layers for baby crying detection project

4.6 Model generation

Once all data has been collected and the preprocessor is built, data needs to be passed to the model for training. The movement type detection and baby crying detection accelerator projects include four model architectures that are ready to be trained with your data.

1. To observe the supplied models, select the **Training** tab, and observe that four models are included.

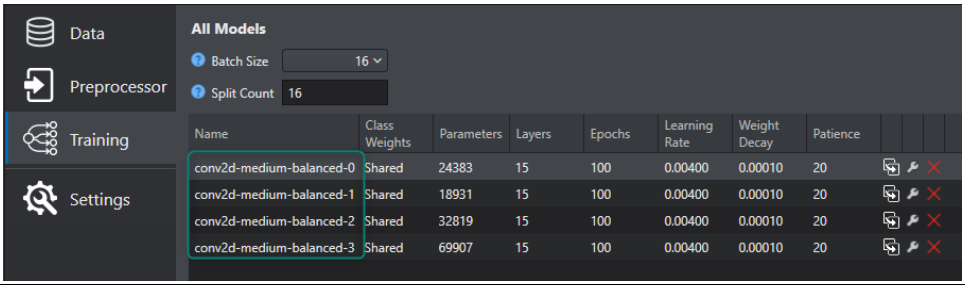
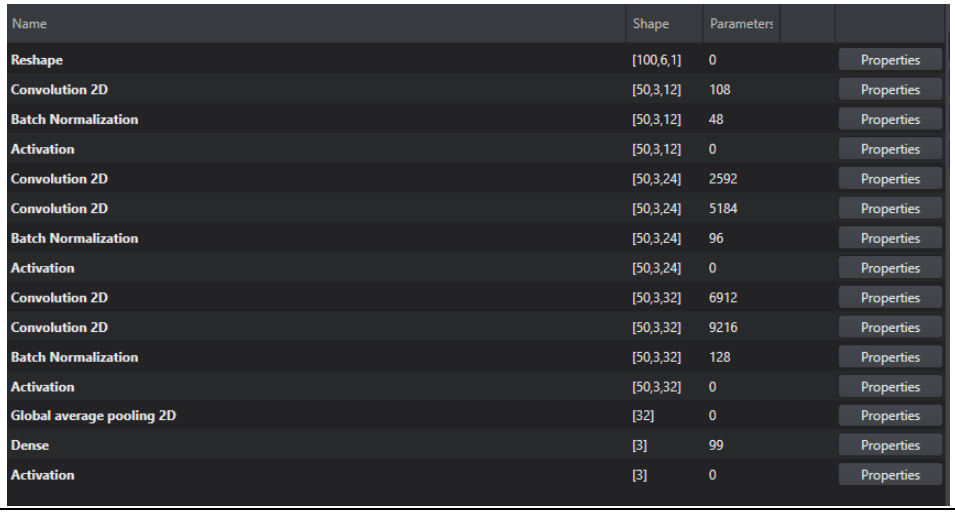


Figure 35 Model selection

2. Select one of the four models generated by DEEPCRAFT™ Studio. This brings up the corresponding model architecture.

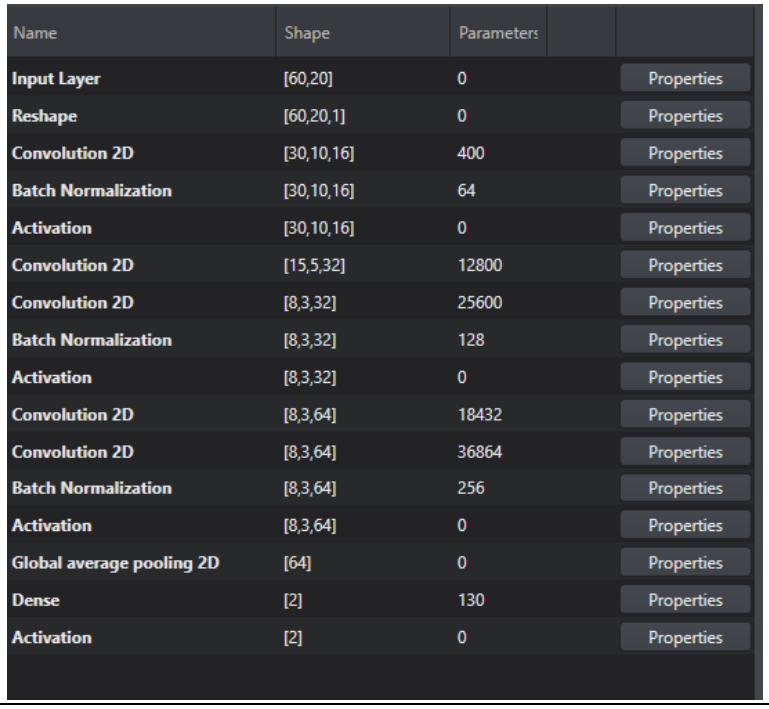
   a) For movement type detection using IMU, the supplied model is a 2D convolutional neural network containing several convolutional layers, batch normalization, activation functions, pooling, and a dense layer for classification into three classes, as shown in the following image:



Name	Shape	Parameters	
Reshape	[100,6,1]	0	Properties
Convolution 2D	[50,3,12]	108	Properties
Batch Normalization	[50,3,12]	48	Properties
Activation	[50,3,12]	0	Properties
Convolution 2D	[50,3,24]	2592	Properties
Convolution 2D	[50,3,24]	5184	Properties
Batch Normalization	[50,3,24]	96	Properties
Activation	[50,3,24]	0	Properties
Convolution 2D	[50,3,32]	6912	Properties
Convolution 2D	[50,3,32]	9216	Properties
Batch Normalization	[50,3,32]	128	Properties
Activation	[50,3,32]	0	Properties
Global average pooling 2D	[32]	0	Properties
Dense	[3]	99	Properties
Activation	[3]	0	Properties

Figure 36 Model layers for movement type detection project

a. For baby crying detection using PDM/PCM, the detection model is a 2D convolutional neural network containing the following layers:



Name	Shape	Parameters	
Input Layer	[60,20]	0	Properties
Reshape	[60,20,1]	0	Properties
Convolution 2D	[30,10,16]	400	Properties
Batch Normalization	[30,10,16]	64	Properties
Activation	[30,10,16]	0	Properties
Convolution 2D	[15,5,32]	12800	Properties
Convolution 2D	[8,3,32]	25600	Properties
Batch Normalization	[8,3,32]	128	Properties
Activation	[8,3,32]	0	Properties
Convolution 2D	[8,3,64]	18432	Properties
Convolution 2D	[8,3,64]	36864	Properties
Batch Normalization	[8,3,64]	256	Properties
Activation	[8,3,64]	0	Properties
Global average pooling 2D	[64]	0	Properties
Dense	[2]	130	Properties
Activation	[2]	0	Properties

Figure 37 Model layers for baby crying detection project

3. Layers can be added or removed by using the plus (+) and X buttons. This is not required for this lab as the models provided are ready for training.
4. Start the model training by selecting **Start New Training Job**.

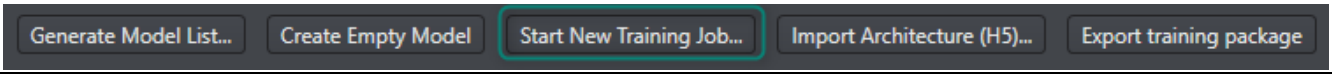


Figure 38 Start new training job

5. If not signed in already, a prompt directs you to the DEEPCRAFT™ account sign-in to start the training.
6. A **New Training Job** window comes up showing the available credits, and a Description window to add comments about the training job. Select **OK** to start the training job.

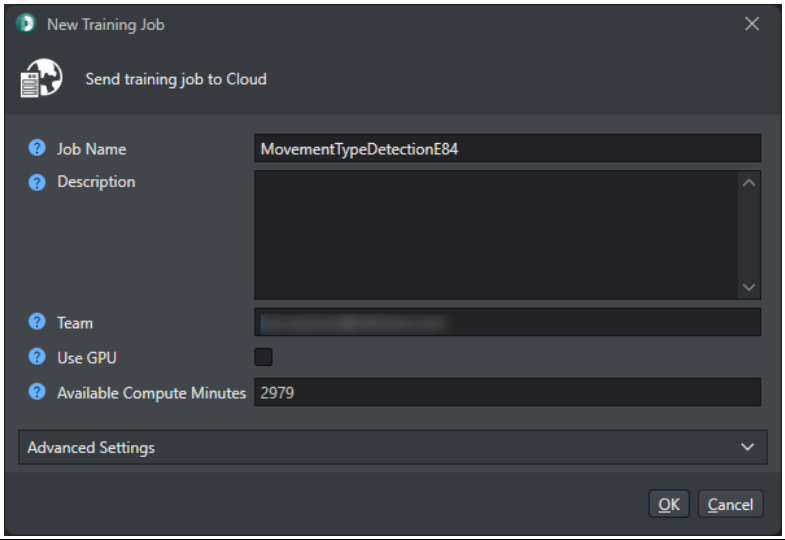


Figure 39 Send training job to cloud

7. Once a training job is started, a window comes up asking if you want to open the current job. Select **Open**. A job screen will show up, showing statistics on the training progress.

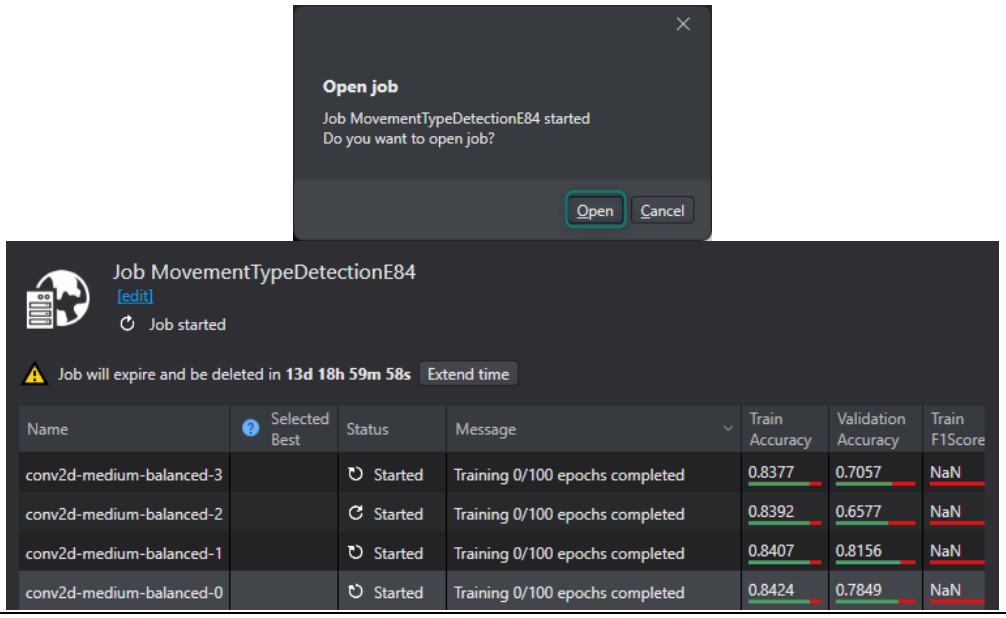


Figure 40 Machine learning job started

Once the **Status** for each model has changed from **Unknown** to **Started** to **Completed**, the model's results can be validated.
Select one of the models, and the training statistics for that model will display.

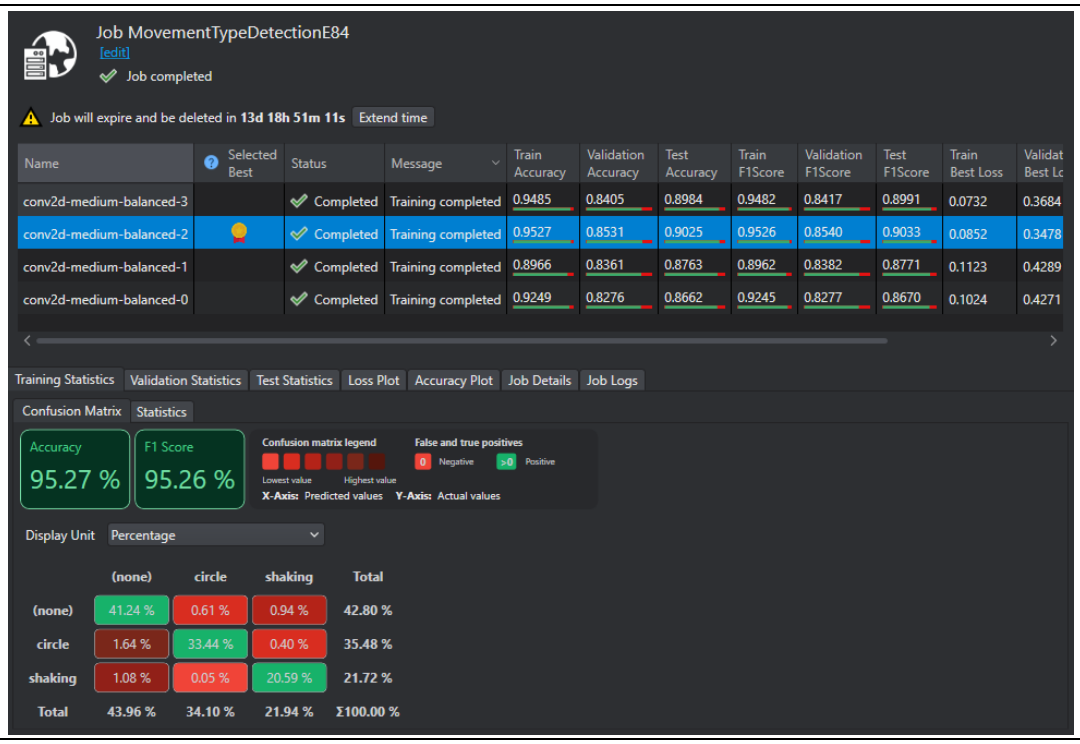


Figure 41 Training results

8. DEEPCRAFT™ Studio compares the model statistics and recommends the best model by putting a gold medal in the **Selected Best** category.

   Note: Model development is an iterative process. If a model does not perform as needed, this may mean that more training data is needed, the preprocessor needs to be reconfigured, or the model layers needs to be changed.

9. When a model meets the user's criteria, the model can be downloaded using the **Download** icon next to the model.

   A **Download Model Files** window comes up. Click on **Download**, select the **Models** folder inside the DEEPCRAFT™ Studio project, then select **OK**.

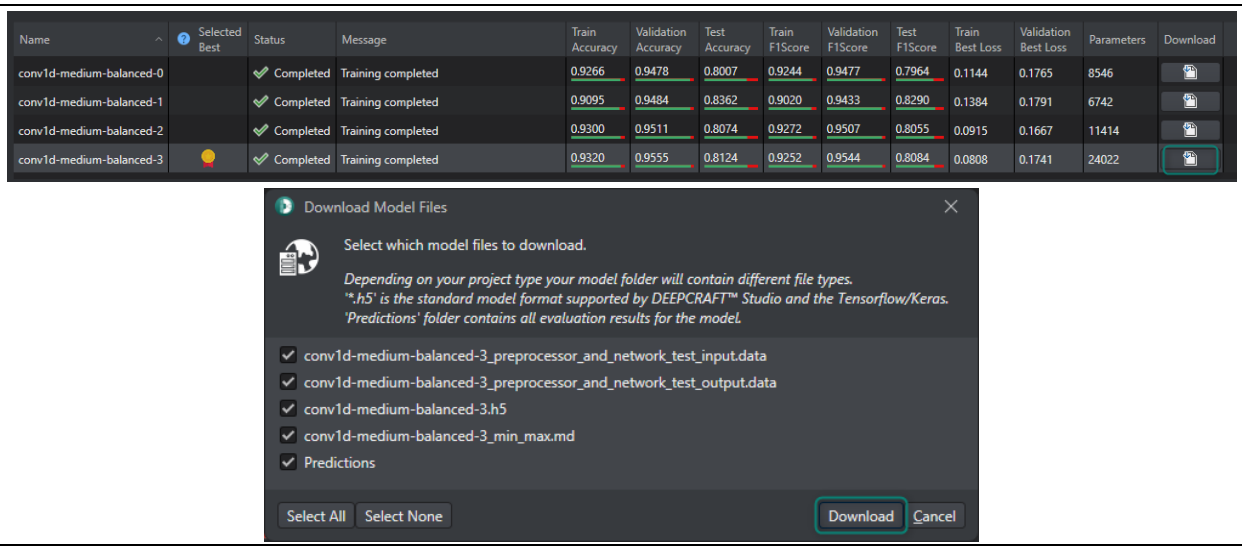


Figure 42 Downloading a model

A model was created and trained with default data from the DEEPCRAFT™ accelerator, and additional data captured using PSOC™ Edge.

4.7 Generate model files

The last step in DEEPCRAFT™ Studio is to generate the machine learning model files.

Along with generating a model, DEEPCRAFT™ Studio generates files for the pre-processing that was selected for the project. To use the preprocessor, three functions are provided:

Table 5 Model functions

Function	Description
IMAI_init	Initializes the preprocessor
IMAI_enqueue	Feeds the preprocessor data
IMAI_dequeue	Returns a buffer with the preprocessed data, which is used to get an inference

To generate the files required for deployment:

1. Following the steps from the previous section, the model should be available in *{DEEPCRAFT™ Studio project}/Models/{model name}*
   Double-click the *.h5 model in this folder.

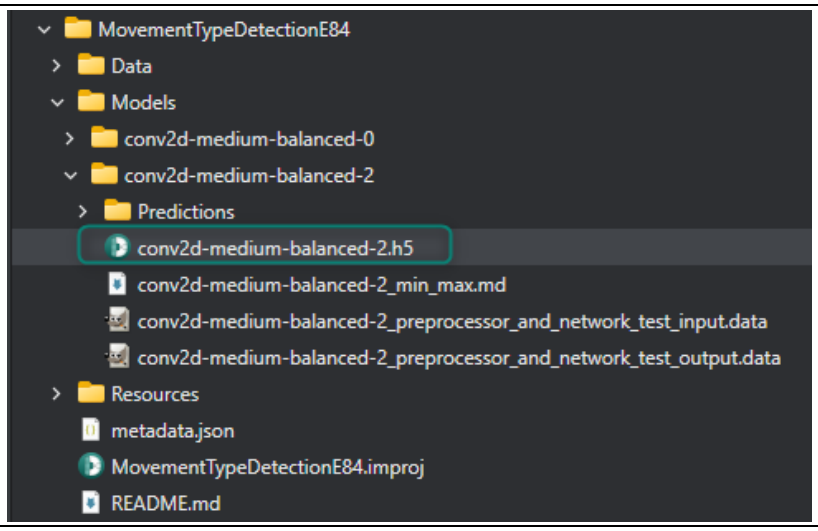


Figure 43 Open .h5 model

2. A new page with model information is displayed.
   Observe that this window includes information about the Model's **Preprocessor**, **Network**, and **Evaluation,** but it also allows to generate code for the model using the **Code Gen** tab.
   Try navigating through the different tabs to observe the model configuration and evaluation results.

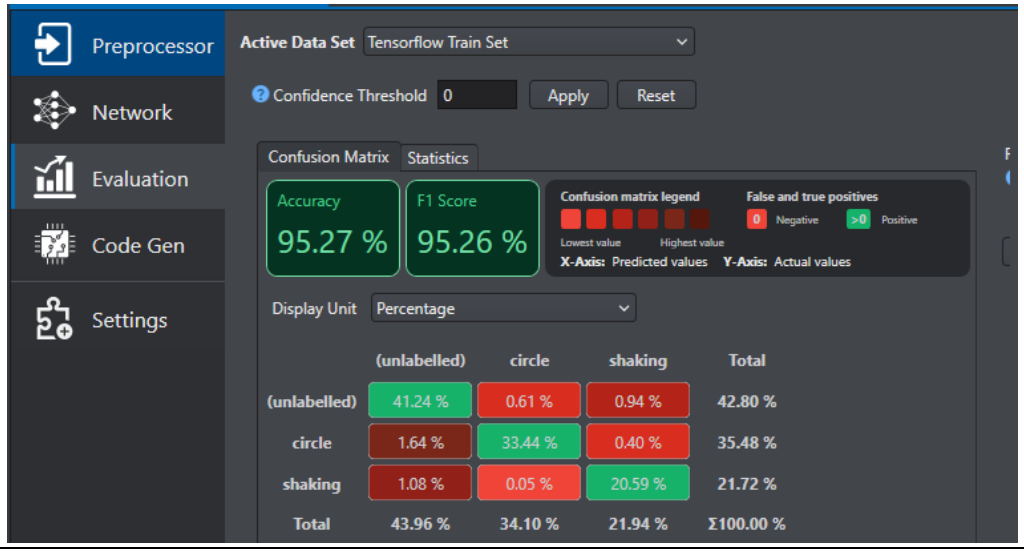


Figure 44 .h5 model window

3. Select the **Code Gen** tab and configure the parameters.
   Note: The Code Gen tab allows you to generate output that can then be used in an embedded device.

Table 6 Code generation for IMU using CM55+U55 with int8x8 quantization

Parameter	Value	Description
Architecture	Infineon PSOC™	Generate code for PSOC™ MCU
Target Device	PSOC™ Edge M55/U55	Generate code for M55 CPU + U55 NPU
Output directory	Infineon	Project folder for output files
Output File Prefix	model	Prefix to match corresponding deploy code example
C Prefix	IMAI_	Prefix to match deploy code example
ModusToolbox™ Project Path	Blank	Files created locally and will be copied manually
Preprocessor Acceleration	CMSIS Floating Point (Float32)	Use CMSIS floating point library
Enable Network Quantization	Enabled	Use 8-bit quantization.
Use Project File	Select corresponding .improj file	Select DEEPCRAFT™ .improj project file for quantization.
Enable Sparsity	Disabled	Sparsity disabled
Skip cleanup	Disabled	Enable cleanup of temporary files
Ethos™-U Tensor Allocator	Hill Climb	Using Hill Climb algorithm for allocating non-constant tensors on the NPU and CPU
Ethos™-U System Config	SOCMEM 300MHz	Using SOCMEM/SystemSRAM at 300MHz
Ethos™-U Optimize	Performance	Performance strategy prioritizes maximal performance by utilizing the specified arena cache memory area size
Ethos™-U Memory Mode	Shared Sram	Shared SRAM is shared between Ethos™-U and Cortex®-M software
Max Block Dependency	3	Maximum value that can be used for the block dependency delay between NPU kernel operations. A lower value may result in longer execution time.

Parameter	Value	Description
Recursion Limit	1000	Python internal limit to depth of recursion. If generation fails with a recursion error, increase the limit using this option.
Arena Cache Size	2936012	Size of the arena cache memory area, in bytes

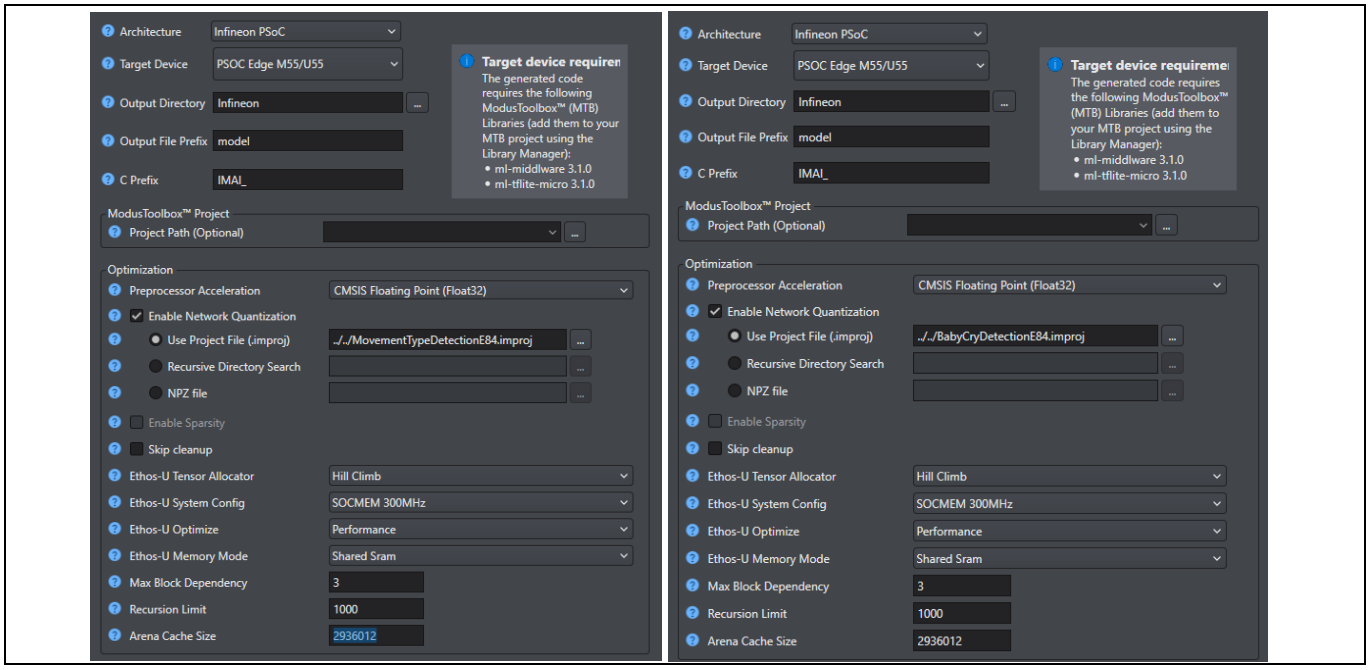


Figure 45 Code Gen parameters for IMU (left) and PDM/PCM (right) projects

4. Select **Generate Code** to generate the source.
   A **Code Generation Report** will appear after completion.
   The generated files will be available under *Models/{model name}/Infineon folder*.

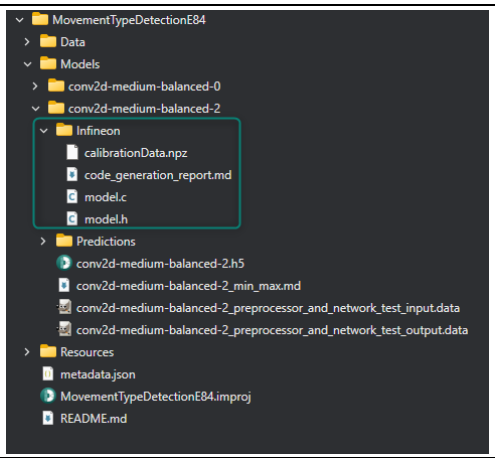


Figure 46 Generated code

4.8 Conclusion

We successfully generated model files with the addition of the previously collected data.

These model files are now ready to be deployed onto a PSOC™ Edge E84 MCU device once we copy them to the ModusToolbox™ deploy project.

5 Code example: DEEPCRAFT™ ML deploy (IMU or PDM/PCM)

5.1 Objective

Now that we have our model created and trained, it's time to deploy it. This section demonstrates how to deploy a model onto a PSOC™ Edge E84 MCU device.

ModusToolbox™ includes deploy examples for IMU and PDM/PCM, and the projects can be configured to run the model on the Arm® Cortex®-M55 CPU leveraging the Arm® Ethos™-U55 NPU (M55/U55), or the Arm® Cortex®-M33 CPU (CM33).

Note: Images and steps in this section are shown for IMU if the process between both flows is similar; otherwise, the guide explains the differences when using PDM/PCM.

5.2 Description

ModusToolbox™ includes the following code examples to deploy ML models:

- PSOC™ Edge Machine learning DEEPCRAFT™ Deploy Motion: for deploying IMU-based movement type detection model
- PSOC™ Edge Machine learning DEEPCRAFT™ Deploy Audio: for deploying PDM/PCM-based baby crying detection model

These examples use the output of DEEPCRAFT™ code generation to deploy the model to the device.

5.2.1 Description of movement type detection project

The **movement type detection** model takes data from the BMI270 motion sensor. The data consists of the 3-axis accelerometer data and 3-axis gyroscope data obtained from the BMI270 IMU at 50 Hz. The MCU receives the data from the IMU once every 200ms using an internal FIFO within the BMI270 IMU, and the IMU then triggers a hardware interrupt so that the PSOC™ Edge MCU can receive the data and perform data processing.

The interrupt handler reads all the data from the sensor via I2C and feeds it to the DEEPCRAFT™ Studio preprocessor using the *IMAI_enqueue()* function. After the preprocessor has enough data, the *IMAI_dequeue()* function returns the results as scores corresponding to each label. All label scores, along with the label corresponding to the maximum score, are printed on the UART terminal.

The pre-processing layer uses a 50-frame sliding window and is shown in Pre-processing.

The model is a 2D convolutional neural network model consisting of several convolutional layers, as shown in Model generation.

5.2.2 Description of baby crying detection project

The **baby crying detection** model processes audio data from the pulse-density modulation (PDM) microphone to detect whether a baby is crying or not.

Input data is received as raw audio from the PDM microphones, then converted to PCM using the built-in PDM/PCM converter block. The received PCM data has a 16kHz sampling rate and it is fed into the audio processing code generated by DEEPCRAFT™ Studio using the *IMAI_enqueue()* function after 1024 samples are received. After the processing is done, the *IMAI_dequeue()* function returns the results in the form of scores corresponding to each of the labels.

All the label scores, along with the label corresponding to the maximum score beyond a threshold, are printed on the UART terminal.

The pre-processing layer is shown in section 4.5.

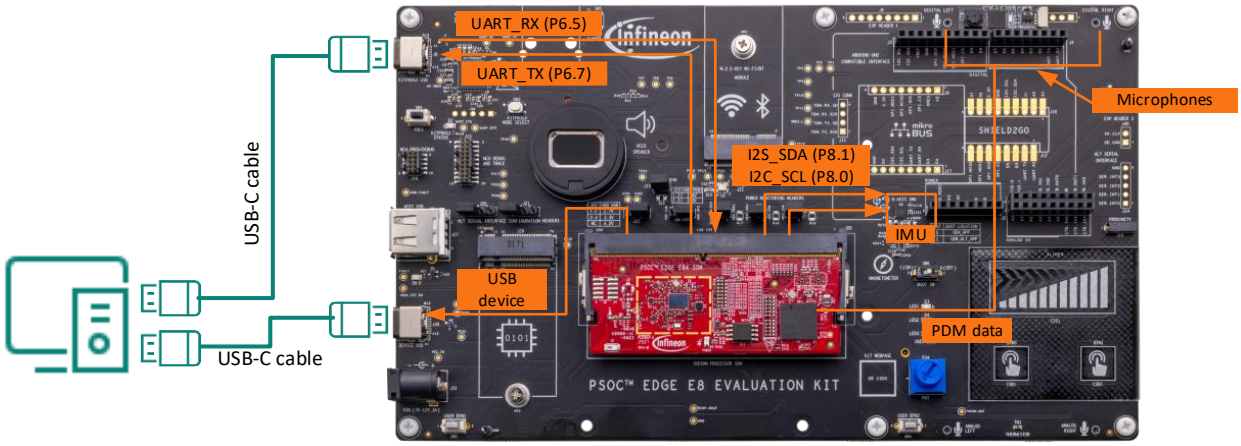
This project uses a 2D convolutional neural network model, as shown in section 4.6.

5.3 Hardware overview

- UART (Transmit data):
  - P6_7 (UART_TX)
  - P6_5 (UART_RX)
- GPIO (User button):
  - P8_3 (USER_BTN1)
- I2C (IMU):
  - P8.1 (SDA)
  - P8.0 (SCL)
- PDM/PCM (Audio):
  - P8_6 (PDM_Data)
  - P8_5 (PDM_CLK)
- USB device

Note: This example requires BOOT SW in ON position.

See Appendix B: KIT_PSE84_EVAL details for more details.



Note: the signals shown are for illustration purposes, not exact routing

Figure 47 Hardware diagram for ML deploy examples

5.4 Project creation

1. Follow steps 1-4 in Appendix A: Creating a PSOC™ Edge application in ModusToolbox™ to create a new application.
2. Select the corresponding application, depending on whether you are using IMU or PDM/PCM
   To prevent issues with Windows path length, it's recommended to rename the project if your worskpace path is long.

a) For IMU, select the PSOC™ Edge Machine Learning DEEPCRAFT™ Deploy Motion application under the **Machine Learning** section.

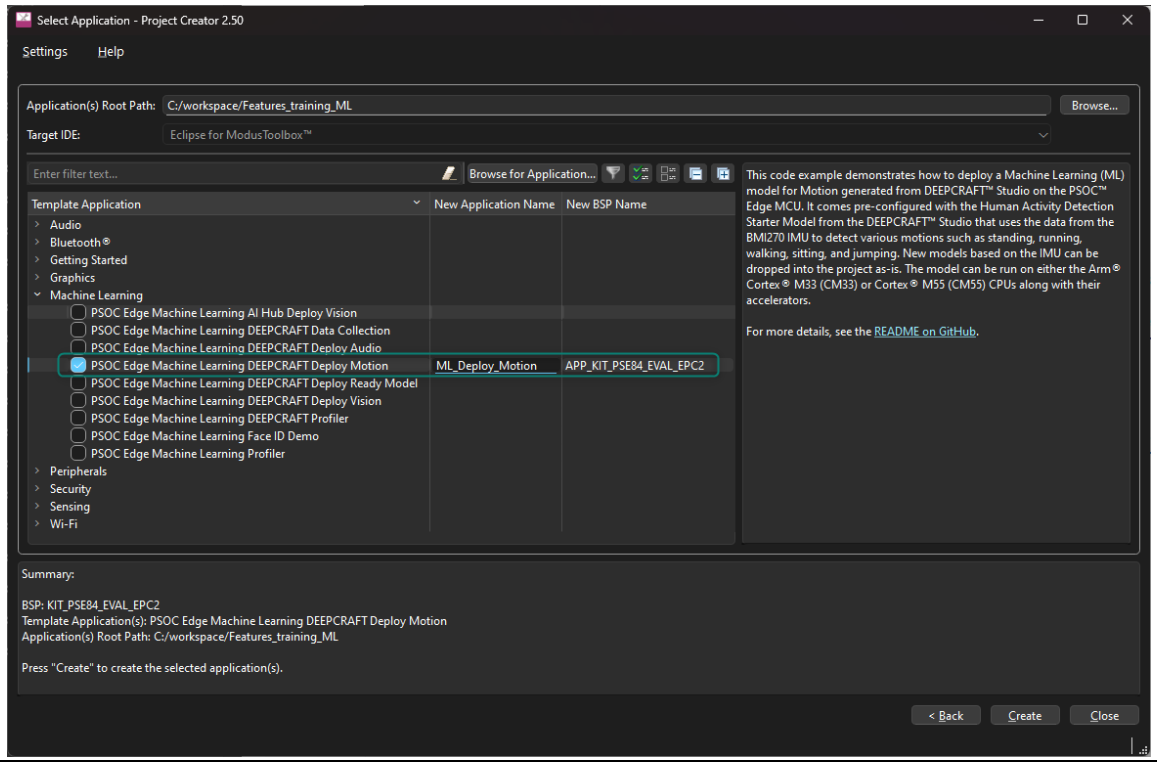


Figure 48 Creating deploy motion project for movement type detection model

b) For PDM/PCM, select the PSOC™ Edge Machine Learning DEEPCRAFT™ Deploy Audio application under the **Machine Learning** section.

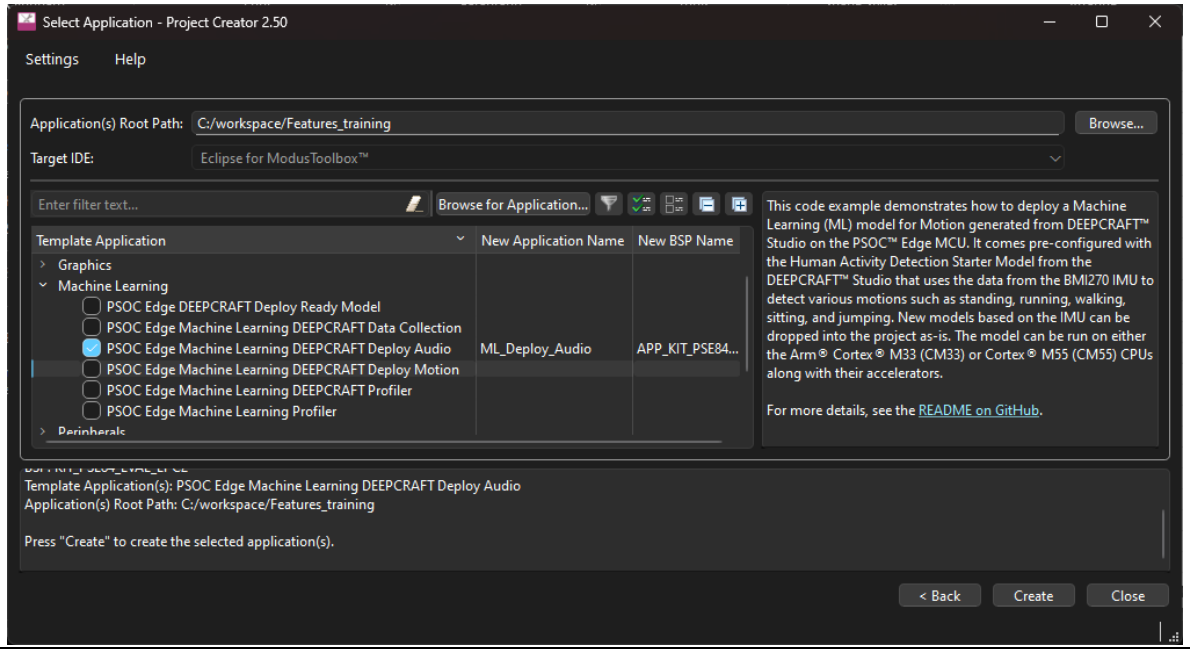


Figure 49 Creating deploy audio project for baby crying detection model

3. Build and program the project.

   Note: First, we will try the default project included in ModusToolbox™, and we will add our updated model in subsequent steps.

5.5 Output

5.5.1 Output using IMU

After programing is successful, open the serial terminal and configure it as 115200-8-N-1.

The confidence for each classification is shown on the terminal. Shake the board or perform a circle motion and see the confidence levels change accordingly.

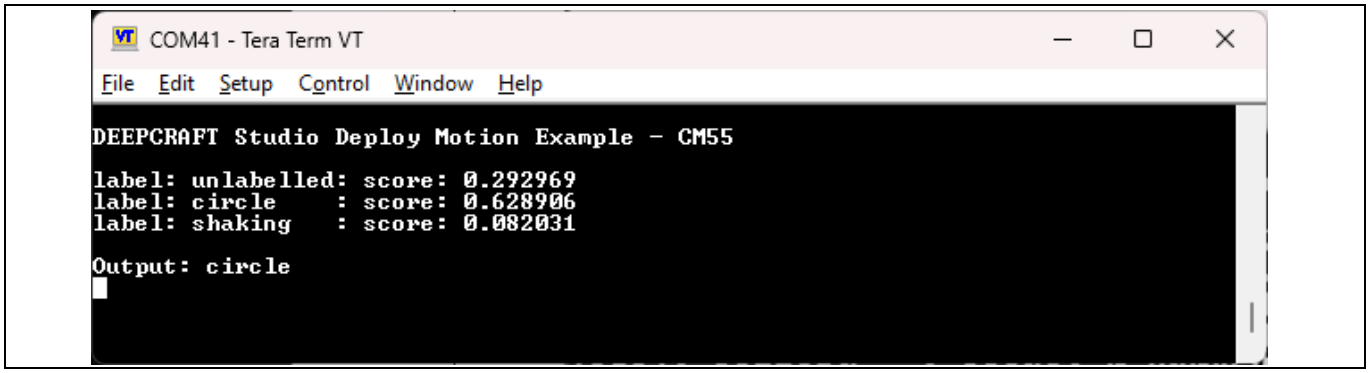


Figure 50 Output of movement type detection example using IMU

5.5.2 Output using PDM/PCM

After programing is successful, open the serial terminal and configure it as 115200-8-N-1.
The confidence for each classification is shown on the terminal.
Use a speaker to play baby-crying sounds and observe confidence levels in the terminal.

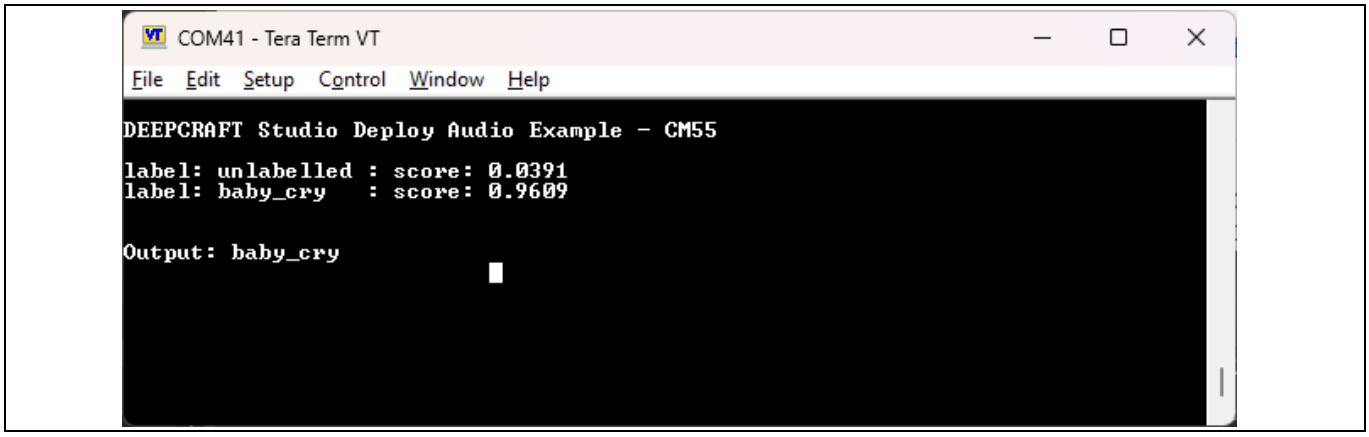


Figure 51 Output of baby crying detection example using PDM/PCM

5.6 Modify the application

The deploy code example comes with corresponding pre-trained model files for movement type detection and baby crying detection. After completing the previous labs, you should have generated the new model files required to deploy your own customized DEEPCRAFT™ model.

To deploy the updated model:

1. Copy the following files from the DEEPCRAFT™ Studio project into the deploy project.

- Source: *{DEEPCRAFT project}/Models/{model_name}/Infineon/*
- Destination: *{ModusToolbox™ Deploy Example}.proj_cm55/model/*
- Files: *model.c/.h*

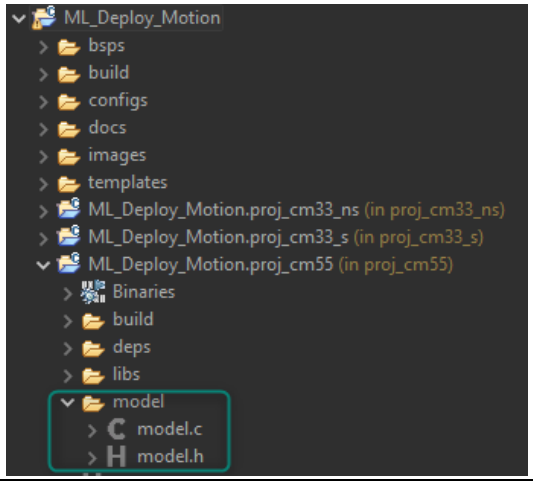


Figure 52 Updating files in ModusToolbox™ deploy example

2. Build and re-program the device with the configured application.
3. Open a terminal program and select the KitProg3 COM port. Set the serial port parameters to **8N1** and **115200** baud.

5.6.1 Further modification

The code examples support running models on both the M33 and M55.

- Open *common.mk* and set ML_DEEPCRAFT_CPU to cm55 or cm33.
- Open the corresponding *proj_cmxx/Makefile* and set NN_TYPE to **int8x8** or float to select desired quantization
- Adjust the **Code Generation** parameters accordingly in DEEPCRAFT™ Studio

5.7 Conclusion

You have now successfully created a PSOC™ Edge E84 MCU code example that runs a model generated in DEEPCRAFT™ Studio.

6 Appendix A: Creating a PSOC™ Edge application in ModusToolbox™

The following steps show how to create a new project for PSOC™ Edge in ModusToolbox™.

4. Open the **ModusToolbox™ Dashboard 3.6** and launch the **Eclipse IDE for ModusToolbox™**.

Note: If you have not installed ModusToolbox™ or Eclipse IDE, see the Prerequisites section.

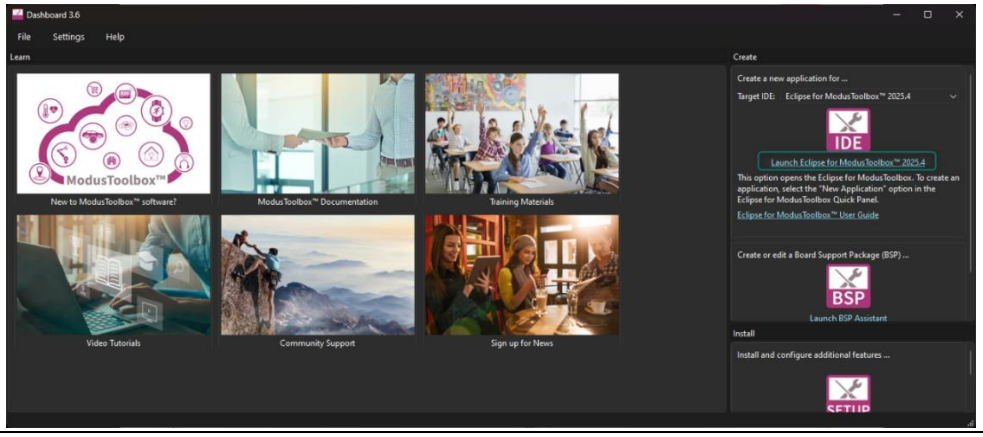


Figure 53 Open Eclipse IDE for ModusToolbox™ from Dashboard

5. Choose a **workspace directory** for your project.
   A folder will be created if the workspace does not exist.
   After selecting a suitable directory, click the **Launch** button.
   Note: It is recommended to keep the workspace path short. Windows has a 260 characters path-length limit and a long workspace path may cause build issues when creating some applications.

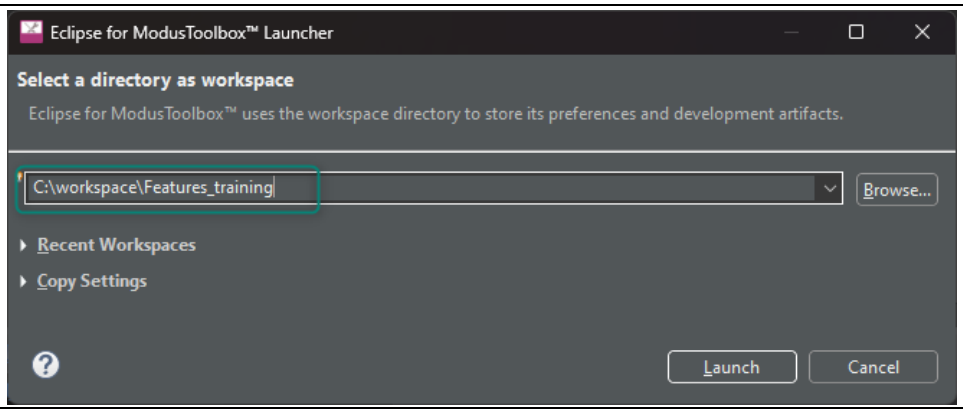


Figure 54 Choose workspace directory

6. Select **New Application** from the Quick Panel. Alternatively, go to **File> New> ModusToolbox™** application. This will launch the **Project Creator Tool**.

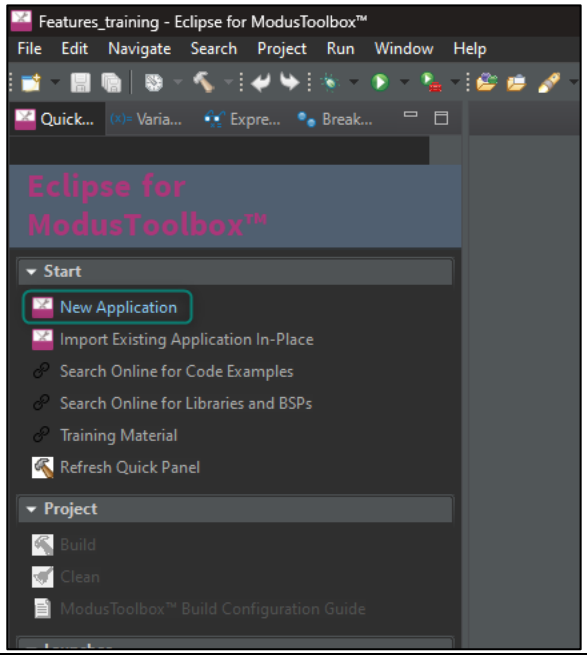


Figure 55 Create new application

7. Once **Projector Creator** launches, you will be prompted to select a board support package (BSP).
   Select the **KIT_PSE84_EVAL_EPC2 BSP** under the PSOC™ Edge BSPs and click **Next**.
   *Note: BSPs are aligned with our development/evaluation kits; they provide files for basic device functionality. A BSP typically has a **design.modus** file that configures clocks and other board-specific capabilities. That file is used by the ModusToolbox™ configurators. A BSP also includes the required device support code for the device on the board. You can modify the configuration to suit your application.*
   Observe that ModusToolbox™ also includes BSPs for PSOC™ Edge EPC4 devices (KIT_PSE84_EVAL_EPC4), and for the PSOC™ Edge AI Kit (KIT_PSE84_AI); however, this training is intended to be used with the KIT_PSE84_EVAL which includes EPC2 devices by default.

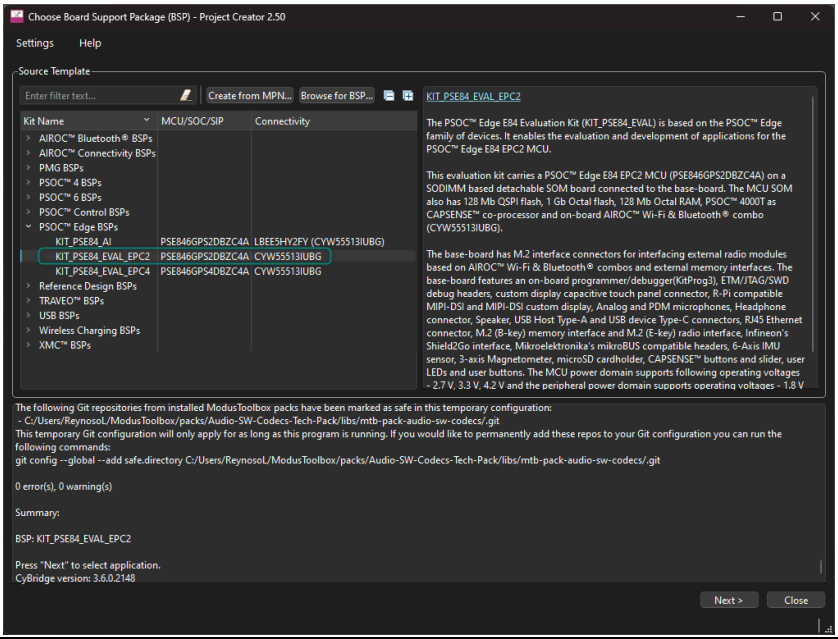


Figure 56 Selecting BSP in Project Creator

8. A new window opens to select an application. The Project Creator includes categories and a search field to make it easier to find and select different examples.

   Click the **checkbox** near the application, optionally rename the project, and then click **Create**.

   Figure 57 shows the creation of the **PSOC™ Edge Hello World** application under the **Getting Started section**.

   Note: Windows has a 260 characters path-length limit. A long workspace path and/or a long project name may cause build issues when creating some applications.

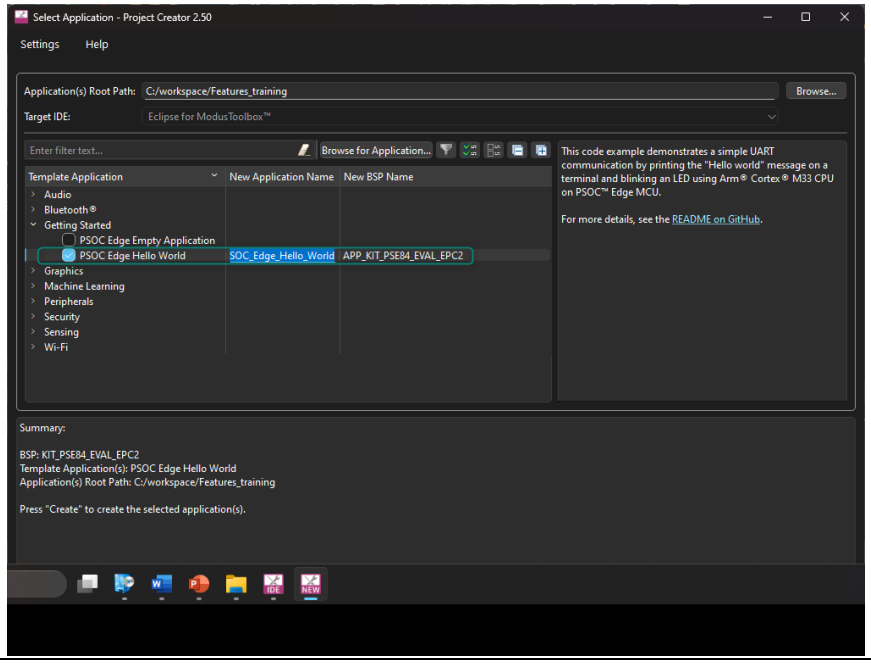


Figure 57 Selecting application

9. A new project is created, and with all the files visible in the **Project Explorer** window.

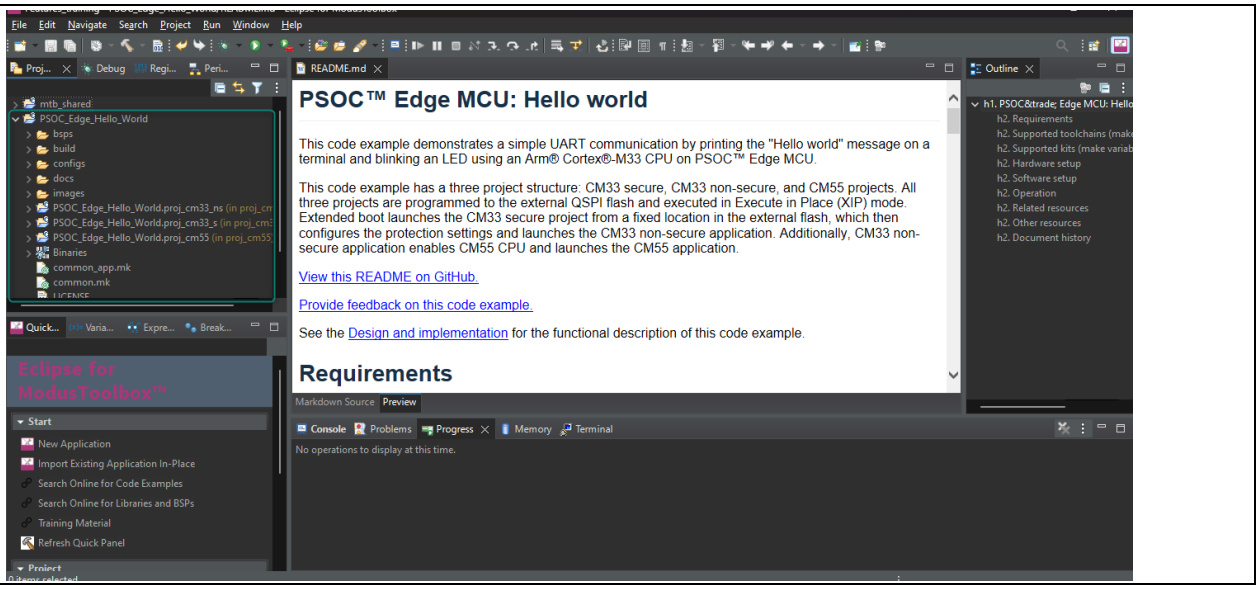


Figure 58 New application created

7 Appendix B: KIT_PSE84_EVAL details

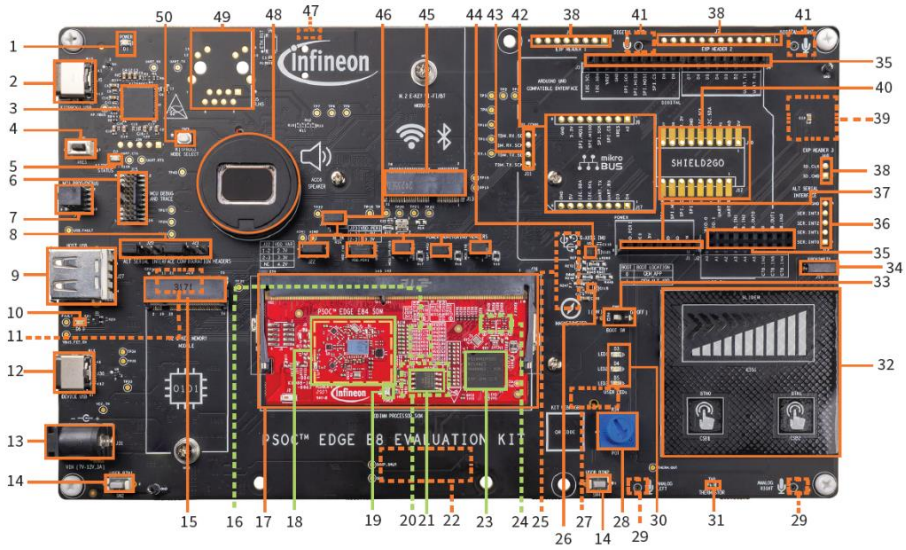


Figure 59 PSOC™ Edge evaluation kit (EVK) details

1 Base board power LED (D1)
2 KitProg3 program/debug USB Type-C connector (J8)
3 PSOC™ 5LP-based KitProg3 programmer and debugger (CY8C5868LTI-LP039, U2)
4 Reset button (SW1)
5 KitProg3 status LED (D2)
6 PSOC™ Edge E84 MCU ETM/JTAG debug and trace header (J15)
7 PSOC™ Edge E84 MCU 10-pin SWD/JTAG program and debug header (J16)
8 Alternative serial interface configuration headers (J20, J21)
9 PSOC™ Edge E84 MCU USB host Type-A connector (J27)
10 USB Type-C Power Delivery (PD) fault LED (D6)
11 Custom display capacitive touch panel connector (J37)**
12 PSOC™ Edge E84 MCU USB device Type-C connector (J30)
13 External power supply VIN connector (J31)
14 PSOC™ Edge E84 MCU user buttons (SW2, SW4)
15 M.2 (B-key) memory interface connector (J29)
16 128-Mb Octal-SPI HYPERRAM™ (S70KS1283GABHI020, U12)***
17 Processor system-on-module (SoM) 260-pin SODIMM connector (J28)
18 AIROC™ CYW55513 tri-band (Wi-Fi & Bluetooth®) combo radio (U3) section
19 Processor system-on-module (SoM) power LED (D3)
20 1-Gb Octal-SPI NOR flash (S28HS01GTGZBH1030, U10)***
21 128-Mb Quad-SPI NOR flash (S25FS128SAGMFB100, U11)
22 MIPI-DSI custom display connector (J38)**
23 PSOC™ Edge E84 MCU (PSE846GPS2DBZC4A, U1)
24 PSOC™ 4000T CAPSENSE™ Co-processor (U9)***
25 Raspberry Pi-compatible MIPI-DSI display connector (J39)**
26 3-axis magnetometer (U4)
27 Raspberry Pi compatible display capacitive touch connector (J41)**
28 Linear potentiometer (R34)
29 Analog microphones (IM73A135V01XTSA1, U36 and U37)**
30 User LEDs (D3, D4, D5)
31 Thermistor (TH1)
32 CAPSENSE™ buttons and slider (CSB1, CSB2, CSS1)
33 BOOT configuration switch (SW6)
34 Proximity sense connector (J19)
35 I/O headers compatible with Arduino UNO R3 (J2, J3, J4)
36 Alternative serial interface I/O header (J14)*
37 Power header compatible with Arduino UNO R3 (J1)
38 PSOC™ Edge E84 MCU expansion I/O headers (J6, J7, J40)*
39 MicroSD card holder (J35)**
40 Infineon Shield2Go interface headers (J10, J12)*
41 PDM microphones (IM72D128V01XTMA1, U7 and U8)**
42 mikroBUS compatible headers by Mikroelektronika (J9, J17)*
43 Extended I2S header (J11)*
44 6-axis accelerometer and gyroscope IMU (U5)
45 M.2 (E-key) radio interface connector (J13)
46 PSOC™ Edge E84 MCU power selection/ monitoring headers (J18, J22, J23, J24, J25, J26)
47 Headphone connector (J34)*
48 Speaker (ACC6)
49 RJ45 ethernet magjack connector (J5)*
50 KitProg3 programming mode selection button (SW3)

* Footprint only, not populated on the board
** Component at the bottom side of the Baseboard
*** Component at the bottom side of the SoM

Revision history

Document revision	Date	Description of changes
**	2026-01-15	Initial release.

Trademarks

All referenced product or service names and trademarks are the property of their respective owners.

PSOC™, formerly known as PSoC™, is a trademark of Infineon Technologies. Any references to PSoC™ in this document or others shall be deemed to refer to PSOC™.

Edition 2026-01-15

Published by

Infineon Technologies AG

81726 Munich, Germany

© 2026 Infineon Technologies AG. All Rights Reserved.

Do you have a question about this document?

Email:

erratum@infineon.com

Important Notice

Products which may also include samples and may be comprised of hardware or software or both ("Product(s)") are sold or provided and delivered by Infineon Technologies AG and its affiliates ("Infineon") subject to the terms and conditions of the frame supply contract or other written agreement(s) executed by a customer and Infineon or, in the absence of the foregoing, the applicable Sales Conditions of Infineon. General terms and conditions of a customer or deviations from applicable Sales Conditions of Infineon shall only be binding for Infineon if and to the extent Infineon has given its express written consent.

For the avoidance of doubt, Infineon disclaims all warranties of non-infringement of third-party rights and implied warranties such as warranties of fitness for a specific use/purpose or merchantability.

Infineon shall not be responsible for any information with respect to samples, the application or customer's specific use of any Product or for any examples or typical values given in this document.

The data contained in this document is exclusively intended for technically qualified and skilled customer representatives. It is the responsibility of the customer to evaluate the suitability of the Product for the intended application and the customer's specific use and to verify all relevant technical data contained in this document in the intended application and the customer's specific use. The customer is responsible for properly designing, programming, and testing the functionality and safety of the intended application, as well as complying with any legal requirements related to its use.

Unless otherwise explicitly approved by Infineon, Products may not be used in any application where a failure of the Products or any consequences of the use thereof can reasonably be expected to result in personal injury. However, the foregoing shall not prevent the customer from using any Product in such fields of use that Infineon has explicitly designed and sold it for, provided that the overall responsibility for the application lies with the customer.

Infineon expressly reserves the right to use its content for commercial text and data mining (TDM) according to applicable laws, e.g. Section 44b of the German Copyright Act (UrhG).If the Product includes security features:

Because no computing device can be absolutely secure, and despite security measures implemented in the Product, Infineon does not guarantee that the Product will be free from intrusion, data theft or loss, or other breaches ("Security Breaches"), and Infineon shall have no liability arising out of any Security Breaches.

If this document includes or references software:

The software is owned by Infineon under the intellectual property laws and treaties of the United States, Germany, and other countries worldwide. All rights reserved. Therefore, you may use the software only as provided in the software license agreement accompanying the software.

If no software license agreement applies, Infineon hereby grants you a personal, non-exclusive, non-transferable license (without the right to sublicense) under its intellectual property rights in the software (a) for software provided in source code form, to modify and reproduce the software solely for use with Infineon hardware products, only internally within your organization, and (b) to distribute the software in binary code form externally to end users, solely for use on Infineon hardware products. Any other use, reproduction, modification, translation, or compilation of the software is prohibited. For further information on the Product, technology, delivery terms and conditions, and prices, please contact your nearest Infineon office or visit https://www.infineon.com